

README

Part 7: 现代 LLM / Minimind — 从 Part 6 的 GPT 到 RoPE、GQA、SwiGLU、DPO

> 把 Part 6 那个"会写莎士比亚的 mini-GPT", 升级成现代大语言模型 (LLM) 的完整形态——最终从零复现 minimind, 一个约 26M 参数的 tiny LLM。

章节导航

	内容
tokenizer.md)	为什么需要 subword、BPE 算法原理、训练 6400 词表、压缩率对比、chat 格式预告
oPE](02_modern_components.md)	LayerNorm 回顾、RMSNorm、RoPE 旋转位置编码、权重绑定
V Cache、MoE](03_gqa_and_ffn.md)	MHA 回顾、GQA/MQA、KV Cache、Flash Attention、SwiGLU、MoE
T → DPO](04_training_pipeline.md)	预训练技巧、SFT + Loss Masking、DPO、完整流水线与部署
5_reproduce_minimind.md)	课程脚本 ↔ 官方 trainer 对照、真实数据下载、四阶段超参、验收与成本；进阶实验：RoPE 外推四件套 + 迷你 RU
](06_attention_mla_nsa.md)	MLA 低秩 KV 压缩、NSA 三支稀疏注意力

前置知识

- 必须掌握：[Part 6 全部内容](../Part6_transformer/tutorial/README.md)（字符级 tokenizer、Dataloader、self-attention、Multi-Head、残差连接、LayerNorm（pre-norm）、decoder-only GPT——Part 7 是在它的骨架上"换零件"）
 - 建议掌握：[Part 3 的 BatchNorm](../Part3_batchnorm/tutorial/02_batchnorm.md)（归一化的思想、可学习的缩放参数 γ / β ——讲 RMSNorm 时会和它对照）；[Part 6 04 章的 RLHF 概念](../Part6_transformer/tutorial/04_beyond_transformer.md)（SFT → 奖励模型 → PPO——DPO 是"消灭 PPO"的替代方案）
 - 可选：[Part 4 的反向传播直觉](../Part4_backprop/tutorial/README.md)（"归一化层里的每个参数都是可学习的"——讲 RMSNorm 去掉 bias 时用到）；[Part 5 的非线性激活](../Part5_wavenet/tutorial/02_wavenet_architecture.md)（tanh——讲 SwiGLU 时拿它和 ReLU/tanh 对比）
- > 如果你卡住了，随时回看前几章的 [tutorial/](#) 目录。Part 7 每一章开头都有「前置知识」告诉你该回看哪。

学习路线图

、

Part 6 (Transformer / GPT：字符级、LayerNorm、learned PE、MHA、ReLU FFN、预训练)

|

| "架构懂了，但 ChatGPT/GPT-3 里真正的 LLM 用的是另一套组件..."



Part 7: 现代 LLM / Minimind (~26M 参数) |

① BPE Tokenizer — 字符级 → 6400 子词 | ———→ 01_bpe_tokenizer.md

② 现代组件 — RMSNorm + RoPE | ———→ 02_modern_components.md

③ GQA 与 FFN — GQA/KV Cache/SwiGLU/MoE | ———→ 03_gqa_and_ffn.md

④ 训练流水线 — Pretrain→SFT→DPO | ———→ 04_training_pipeline.md

⑤ 复现毕业指南 — 官方仓库对照/进阶实验 | ———→ 05_reproduce_minimind.md

⑥ 注意力演进 — MLA + NSA | ———→ 06_attention_mla_nsa.md

"现在你拥有一个完整的现代 LLM 全家桶..."

▼

等价于 minimind（从零复现完成）

数据与依赖

本课程完全自包含：数据、权重都不需要提前下载，全部脚本可直接跑通——

西	说明
依赖	<code>`data/input.txt`</code> （tiny Shakespeare）已在仓库内，脚本 01 – 11 都用它（含 09 三阶段验收、10 MoE、11 RoPE 外推）；13 用合成 KV 检索任务（无需数据）
权重	仅脚本 01 的「真 BPE」需要 <code>[`tokenizers`](https://pypi.org/project/tokenizers/)</code> （已声明在 <code>`requirements.txt`</code> ）；未安装时自动回退字符级分词
重	不需要 —— 所有权重（分词器 <code>`temp/bpe_tokenizer.json`</code> 、 <code>`ckpt_*.pt`</code> ）都由脚本从零训练并自动生成

可选：对照 minimind 官方中文数据与权重（想用真实中文语料复现原版、或直接加载官方模型时）：

	说明
<code>gonggy/minimind_dataset/files</code> / [HuggingFace](https://huggingface.co/datasets/jingyaogong/minimind_dataset/tree/main)	最小复现只需 <code>`pretrain`</code> 数据
<code>gonggy/minimind-3-pytorch</code> / [HuggingFace](https://huggingface.co/jingyaogong/minimind-3-pytorch)	官方已训好的各尺寸模型
<code>collections/MiniMind-b72f4cfb74b47</code> / [HuggingFace 合集](https://huggingface.co/collections/jingyaogong/minimind-66caf8d999f5c7fa64f399e5)	兼容 HF 生态，可直接加载

``bash``

下载 PyTorch 权重（两种方式任选其一）

```
modelscope download --model gonggy/minimind-3 --local_dir ./minimind-3
git clone https://huggingface.co/jingyaogong/minimind-3
```

> ⚠ 官方权重依赖 minimind 仓库的模型/分词器代码结构，与本课程从零实现的脚本不完全等价，仅作参考学习，不要与本课程脚本混用。

学完这一部分你能...

- 手写/训练一个 BPE tokenizer，把字符级（65 词表）升级到 6400 子词，理解压缩率与 OOV 的取舍
- 从零实现 RMSNorm，讲清楚"为什么现代 LLM 抛弃了 LayerNorm 的均值中心化和 bias"
- 从零实现 RoPE（旋转位置编码），理解"旋转正交、相对位置影响内积、可外推"
- 理解 权重绑定 (tie_word_embeddings) 为什么能让 embedding 层"不花参数"
- 从零实现 GQA（分组查询注意力）和 repeat_kv，讲清"为什么 KV 头可以比 Q 头少"
- 实现 KV Cache，理解生成时为什么只算最后一个 token、复用什么
- 从零实现 SwiGLU FFN（gate/up/down 三投影），对比 ReLU FFN
- 理解 MoE（混合专家）的概念、路由器和负载均衡损失
- 跑通 Pretrain → SFT → DPO 完整流水线，讲清 loss masking、Bradley-Terry、参考模型冻结
- 用 PI / NTK / YaRN 给 RoPE 做长上下文外推（含 YaRN 温度因子 $\sqrt{(1/t)=0.1 \cdot \ln(s)+1}$ ），并用迷你 RULER 的 needle 检索引擎验证"读得顺 ≠ 记得住"
- 对照 minimind 的 train_tokenizer → train_pretrain → train_full_sft → train_dpo 全流程

演进路线：从"迷你 GPT"到"现代 LLM"

本教程会反复看到这张表——Part 6 到 Part 7，换的不是架构骨架，而是每一层零件：

维度	Part 6 的 mini-GPT	Part 7 的现代 LLM（minimind）				
Tokenizer	字符级，65 词表	**BPE**，6400 词表				
归一化	LayerNorm（mean/var + γ/β + bias）	**RMSNorm**（只算均方根，去 bias）				
位置编码	learned positional embedding（可学习参数表）	**RoPE**（旋转编码，零参数、可外推）				
注意力	MHA，每头独立 K/V	**GQA**，8 Q 头 / 4 KV 头				
KV Cache	无	**有**（推理加速）				
FFN	Linear → ReLU → Linear（4×）	**SwiGLU**（gate/up/down， $\sim 3.2\times$ ）				
特殊 token	无	`<\`	im_start\`	>` / `<\`	im_end\`	>` chat 格式
训练	纯预训练	**预训练 → SFT → DPO**				
参数量	~10M	**~26M**				

预训练阶段的损失数字和 Part 6 不可直接比较——词表从 65 变 6400，初始 loss 反而更高（ $\ln 6400 \approx 8.8$ vs $\ln 65 \approx 4.2$ ）；但模型更强、且子词比字符更好预测，收敛后的 per-token loss 往往略低于字符级（ ≈ 2.0 vs 2.23）。看表请抓住"趋势"而不是硬比数字：

阶段	目标	预期损失（CPU 缩小版， $\approx$ ）
Part 6 最终（字符级 GPT）	预测下一个字符	val loss ≈ 2.23
Part 7 预训练（BPE，~26M）	预测下一个 token	val loss ≈ 2.0 , ppl $\approx 7\sim 12$
Part 7 SFT	只对 assistant 回答算 loss	微调 loss 下降、对话变"像话"
Part 7 DPO	让回答更"讨喜"	偏好奖励上升、dpo loss 下降

> △ 我们的脚本是 CPU 缩小版（更小的 hidden/dim、更少的步数、更短的上下文）。不同超参、不同随机种子，数字都会有差异，所以都带 $\approx$ 。看趋势，别死记数字。GPU 全量版请参考 minimind 仓库的超参。

课后作业

每一章末尾有 2-3 道思考题（<details> 折叠答案）。全部学完后，去[这里](#)做动手练习：

相关资源

- [minimind](https://github.com/jingyaogong/minimind) — 本部分复现的目标项目，一个约 26M 参数的 tiny LLM
- Sennrich et al. 2016 : [Neural Machine Translation of Rare Words with Subword Units (BPE)](https://arxiv.org/abs/1508.07909)
- Zhang & Sennrich 2019 : [Root Mean Square Layer Normalization (RMSNorm)](https://arxiv.org/abs/1910.07467)
- Su et al. 2021 : [RoFormer: Enhanced Transformer with Rotary Position Embedding (RoPE)](https://arxiv.org/abs/2104.09864)
- Ainslie et al. 2023 : [GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints](https://arxiv.org/abs/2305.13245)
- Shazeer 2020 : [GLU Variants Improve Transformer (SwiGLU)](https://arxiv.org/abs/2002.05202)
- Shazeer et al. 2017 : [Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer (MoE)](https://arxiv.org/abs/1701.06538)
- Rafailov et al. 2023 : [Direct Preference Optimization: Your Language Model is Secretly a Reward Model (DPO)](https://arxiv.org/abs/2305.18290)
- Llama 2 论文 (2023) : [Llama 2: Open Foundation and Fine-Tuned Chat Models](https://arxiv.org/abs/2307.09288) — 现代 LLM 组件的工业级集大成者

[← 上一章 : Part 6 Transformer](../../Part6_transformer/tutorial/README.md)

01_bpe_tokenizer

01 — BPE Tokenizer : 从 65 个字符到 6400 个子词

> 现代 LLM

的第一块拼图：用字节对编码（BPE）把莎士比亚从"一字符一整数"升级成"一子词一整数"，顺便预告 chat 格式。

前置知识

- 必须掌握：[Part 6 01 章](../../Part6_transformer/tutorial/01_data_and_tokenizer.md)（字符级 tokenizer 的 **encode/decode**、**vocab_size**、**block_size**、train/val 划分）
 - 建议掌握：[Part 6 04 章](../../Part6_transformer/tutorial/04_beyond_transformer.md) 的核心权衡讨论（词表大小 vs 序列长度——BPE 就是它的折中解）
 - 可选：Unicode/字节级编码常识
- > 如果你忘了"为什么字符级序列很长、词级会 OOV"，先回 Part 6 的 **01_data_and_tokenizer.md**。

从 Part 6 结束的地方出发

Part 6 我们用的 tokenizer 是最简单的字符级：把文本里的 65 个唯一字符（换行、空格、标点、大小写字母）做成一张词表，一个字符一个整数。

"To be, or not to be" → encode → [1, 58, 33, 46, 43, 56, 43, 58, 1, 45, 43, 58, 58, 33, 46, 43, 56, 43]
65 个词表里的整数，一个字符一个

这个方案简单，但有一个明显代价：序列很长。一句话四五十个字符，就是四五十个整数；整个莎士比亚 111 万字符，就是 111 万个 token。模型要一步步"吞"这么多 token，训练慢、上下文也覆盖不了多少真实内容。

这一章我们把 tokenizer 升级成 BPE（Byte Pair Encoding，字节对编码）——现代 LLM（GPT、Llama、Qwen）几乎都在用的方案。目标是：词表变大到 6400，但序列大幅变短。

为什么需要 subword tokenizer？

先看三个候选的"粒度"，它们正好构成一个三难问题：

词级（word） 字符级（character） subword（子词） ★

词表几万~几十万 词表很小（65） 词表几千~几万
序列很短 序列超长 序列中等

OOV：没见过的词 任何文本都能编 任何文本都能编
直接崩掉 一个词要拆成一串字符 高频片段被合并，低频拆开

- 词级：词表太大，而且有未知词（OOV）问题——"ChatGPT"、"quoth"这类词训练时没见过，就编不了。
- 字符级：没有 OOV，但"一个词 = 一串字符"，序列太长，模型要花很多步才能"读懂"一个词。
- subword（子词）：介于两者之间——高频的常见片段（如 **ing**、**the**、**tion**）被合并成独立的 token，低频的罕见词则被拆成更小的子词。既没有 OOV，序列又比字符级短得多。

> subword 的核心思想：常见的组合合并成整体，罕见的词退化成字符组合。任何词都能被编码（没有 OOV），常见的词只占 1~2 个 token（序列短）。BPE 就是自动做这件事的算法。

BPE 算法原理：一句话版本

> 从字符集出发，反复合并出现频率最高的相邻 token 对，直到词表达达到目标大小。

听起来抽象，我们用一个玩具例子走一遍。假设文本只有一句：**low low low low low low lower lower**（7 个 low + 2 个 lower），目标词表 10。

第 0 步：初始词表 = 全部字符

['l', 'o', 'w', 'e', 'r']（5 个，去掉重复）

文本：low_low_low_...（每个字符一个 token）

第 1 步：统计相邻 token 对出现的次数

('l', 'o') 出现 9 次 ← 最多！

('o', 'w') 出现 9 次

合并 ('l', 'o') → 新 token 'lo'

词表：['l', 'o', 'w', 'e', 'r', 'lo']（6 个）

第2步：重新统计相邻对

('lo','w') 出现9次 ← 最多！

合并 ('lo','w') → 新 token 'low'

词表：['l','o','w','e','r','lo','low'] (7个)

第3步：('low','er')? 统计相邻对

('low','e') ... ('e','r') ...

继续合并出现最多的，直到词表达到目标10个

- 每一步都问同一个问题：当前文本里，哪两个相邻 token 一起出现的次数最多？把它们合并成一个新的 token。重复，直到词表够大。
- 注意 l、o、w 这些单字符永远留在词表里（作为“最底层”），所以任何词哪怕从没合并过，也能用字符拼出来——这就是“没有 OOV”的保证。
- △ 合并是贪心的：每一步只合并当下最频繁的对，不管未来。这不能保证“全局最优压缩”，但足够好用，而且实现简单。

完整手推一遍：一个更真实的合并过程

上面那个例子只展示到第3

步，容易让人误以为“合并就几下”。真实的合并往往要连续几十上百步。我们用语料 `aaabdaaabac` 推一遍（目标是直观感受“合并表”怎么一步步长出来，不追求跑完整个词表）：

语料：`aaabdaaabac`

初始词表：['a','b','c','d']

第1步：统计相邻对 → ('a','a') 出现4次最多 → 合并成 'aa'

文本：`aaabdaaabac`（注意aaa合并成"aa"+"a"）

第2步：重新统计 → ('a','b') 出现2次最多 → 合并成 'ab'

文本：`aaabd aaabac`

第3步：重新统计 → ('aa','ab') 出现2次最多 → 合并成 'aaab'

文本：`aaabd aaabac`

第4步：剩下所有相邻对都只出现1次（并列）→ 取最靠前的 ('aaab','d') → 'aaabd'

文本：`aaabd aaabac`

...

- 观察三点：① 合并顺序完全由统计驱动（`aaab` 因为反复出现被合并出来）；② 每一步都在更新相邻对统计（合并会创造新的相邻对，比如第2步之后才出现 `aaab` 这个候选）；③ 词表从4个字符慢慢长到目标大小。真实 BPE 就是在百万字符上把这个过程重复几千次。
- △ 两个容易忽略的细节：① 重叠的处理——`aaa` 合并时只取前两个 `a` 成 `aa`，剩下一个 `a` 单独留下（合并是“从左到右、不重用”的）；② 平局的处理——并列时取最靠前/字典序最小的对，不同实现可能有细微差异，但结果都差不多。
- 这个例子还能看出：BPE 完全不管语义——`aaab` 在人类眼里是乱码，但在数据里高频出现，就会被合并。合并的唯一标准是统计频率，不是词义。

用 HuggingFace tokenizers 训练 BPE

自己手写 BPE 完全可以（上面的玩具例子就是原理），但工程上我们直接用 HuggingFace 的 `tokenizers` 库来训练。注意：我们只“借”训练器，模型还是我们自己的。

[01_bpe_tokenizer.py](../scripts/01_bpe_tokenizer.py) 的核心代码：

```
`python
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.pre_tokenizers import ByteLevel
from tokenizers.trainers import BpeTrainer
```

1. 指定模型：ByteLevel BPE（GPT-2 同款，从 UTF-8 字节出发，天然无 OOV）

```
tokenizer = Tokenizer(BPE(unk_token="<|endoftext|>"))
tokenizer.pre_tokenizer = ByteLevel(add_prefix_space=False)
```

2. 指定训练目标：词表 6400，预留 3 个特殊 token 的"坑"

```
trainer = BpeTrainer(
    vocab_size=6400,
    special_tokens=["<|endoftext|>", "<|im_start|>", "<|im_end|>"],
)
```

3. 在 data/input.txt 上训练

```
tokenizer.train(files=[data_path], trainer=trainer)
```

4. 编码 / 解码 / 保存

```
tokenizer.save(model_path) # 存成 tokenizer.json`
```

- 三个关键参数：**vocab_size=6400**（目标词表大小）、**special_tokens=...**（特殊 token 提前占坑）、**ByteLevel**（从 UTF-8 字节出发，保证任何输入都能编）。
- ⚠ 特殊 token 必须在训练时就用 **special_tokens** 预留，否则训练器会把它们当普通文本吃掉，词表里就腾不出它们的固定位置了。后面讲 chat 格式时会看到它们多重要。

三种工业实现对照：HF tokenizers / tiktoken / sentencepiece

课程里会出现两种 BPE 工具，别混淆——BPE 是算法，下面这些是不同定位的实现：

	HF `tokenizers`（本课 Part 7 用）	`tiktoken`（Part 8 用）	`sentencepiece`（Llama 系用）
能力	**能训练**新词表 + 能推理	只能**使用**已发布的词表（GPT-2/3.5/4）	能训练（BPE/Unigram）+ 推理
本课用途	从 Shakespeare 训 6400 词表	直接加载 GPT-2 的 50304 词表	（未使用，认识即可）
词表	自己定（6400）	固定（50257/50304）	自己定
特色	ByteLevel 预分词，任何字符串可编	Rust 实现、极快的编码	把空格变成 `__`，"词首"信息内建

- 为什么 Part 7 训、Part 8 用现成的：Part 7 的重点是"词表从 0 训出来"（亲眼看合并过程与压缩率）；Part 8 走 GPT-2 生命周期，直接沿用它的词表才能对上 50304 的 logits 形状。
- 同一段文本在不同词表下压缩率不同——Part 6 字符级 1 字符/token，本课 BPE 约 4-5 字符/token。比较两个模型的 ppl 前先比较 tokenizer（不同词表的 ppl 不可比，详见 Part 8 07 章）。

字节级 BPE 与预分词：为什么"任何词都能编"

ByteLevel 两个词拆开理解：

- Byte（字节）级：编码时先把每个字符映射成 UTF-8 字节（0~255），BPE 在字节序列上做合并。因为字节只有 256 个，再偏门的字符（emoji、中文、任何语言）都能用字节拼出来——这是"没有 OOV"的最终保证。GPT-2 系列正是这么做的。
- 预分词（pre-tokenizer）：在跑 BPE 之前，先按空格把文本粗切成"词块"。这样 **the**、**and** 这类词天然被完整保留，BPE 只需要在词块内部和少量跨词块边界做合并。

原文本: "To be or not to be"

预分词: ["To", "be", "or", "not", "to", "be"] ← 按空格切，保留空格前缀

字节化: 每个词块 → UTF-8 字节序列

BPE 合并: 在字节/字符片段上反复合并高频对 → 最终 token 序列

- 预分词对英文意义重大：没有它，空格会被当成普通字符混进合并，**the** 可能被拆成 **t+he** 甚至更碎。有了它，整词完整保留，BPE 只负责把词内和常见词组（如 **ing**、**ion**）也压缩掉。
- ⚠ 注意 ByteLevel 的 **add_prefix_space**：训练时如果是 **False**，那么"句首的 To"和"句中 the 前面的空格"处理会有细微差别——这类细节会影响 token 分布，但对我们的教程结果影响很小。

在莎士比亚上：BPE 合并出了哪些子词？（预期输出）

在 110 万字符的莎士比亚上跑 BPE，词表 6400，最靠前（合并最成功、最频繁）的子词大致是（具体 id 因种子而异）：

—— 高频子词示例（前 12 个，≈） ——

Ġthe Ġand Ġof Ġto Ġa Ġin

Ġthat Ġis Ġfor Ġmy ing Ġwith

（Ġ 表示"前面带空格"）

- 看到规律了吗：最高频的 token 都是带空格的整词（**Ġthe**、**Ġand**），其次是常见后缀（**ing**、**ed**、**ion**），再往后是更小的片段。这就是"高频合并成整体、低频退化成碎片"的直接体现。
- 这也解释了压缩率从哪来：莎士比亚里 **the**、**and** 出现几千次，每次都只占 1 个 token（字符级要 3 个）。常见词越多，压缩越狠——英文文本平均能压到 1/3 左右，正是我们看到的 $\approx 3.5\times$ 。

运行结果（预期输出）

实跑脚本（训练在 CPU 上大约十几秒），输出大致如下：

—— BPE Tokenizer 训练 ——

词表大小: 6400

特殊 token: <|endoftext|>(0), <|im_start|>(1), <|im_end|>(2)

训练数据: data/input.txt (1,115,394 字符)

—— 编码演示 ——

encode('To be or not to be') =

[3876, 509, 573, 4824, 3876, 509] ← 6 个 token !
decode(...) = 'To be or not to be'
往返一致: True

———— 压缩率 ————

字符数: 1,115,394
token 数: ≈ 320,000 ← 约 3.5 字符/token
压缩率: ≈ 3.5×

- 同一个句子，字符级要 19 个整数，BPE 只要 6 个——To be、or、not 这些常见片段都成了独立 token。模型每"看"一个 token 的信息量变大，上下文覆盖的真实内容就多了。
- △ 不同种子/训练轮次，具体 token id 会不同（比如 3876 可能变别的数），但数量级不变：6400 词表、约 3.5 倍压缩。

怎么检验一个 tokenizer 好不好

训练完别急着用，先跑三道"体检"：

1. 往返一致性：`decode(encode(s)) == s` 对任意输入都要成立。这是最底线的正确性检查（Part 6 我们讲过，BPE 同样适用）。
2. 覆盖性：把整个训练集重新编码一遍，确认没有产生 unk token（ByteLevel 下理论上不可能，但值得确认）。
3. 压缩率：`字符数 / token 数`。英文通常 3~4；如果只有 1~2，说明词表太小或预分词配置不对；如果高于 4~5，可能词表过大（对 6400 词表而言压缩过头反而说明词表浪费）。

python

脚本里的体检部分

```
total_chars = len(text)
total_tokens = len(tokenizer.encode(text).ids)
print(f"压缩率: {total_chars / total_tokens:.2f}x") # 预期 ≈ 3.5
```

- 这三项对应三个不同层面的问题：正确性（往返）、健壮性（无 unk）、效率（压缩率）。以后你用任何 tokenizer，都值得先做这三道体检。

与字符级的对比：65 vs 6400

维度	Part 6 字符级	Part 7 BPE
词表大小	65	**6400**
编码 "To be or not to be"	19 个整数	**6 个整数**
压缩率	1×	≈ 3.5×
OOV	无	无（ByteLevel 从字节出发）
训练开销	0（直接 set）	需要跑一次 BPE 训练
模型输入维度	65	6400

- 词表变大，意味着 `nn.Embedding(vocab_size, hidden)` 和最后的 `lm_head (hidden → vocab_size)` 都会变大——这是模型参数增加的一个来源。但换来的是序列变短、上下文变长，整体收益远大于开销。

- 模型代码几乎不用改：Part 6 的 Transformer 只依赖 `vocab_size` 这个数。把 65 换成 6400，训练代码一行不用动。这正是"tokenizer 与模型解耦"的好处——你可以在不改模型的情况下，随便换 tokenizer。

特殊 token 与 chat 格式：预告

词表里除了普通子词，我们还预埋了 3 个特殊 token。它们的 id 是词表最前面的几个：

```
<|endoftext|>(id 0) 文本结束 / 填充
<|im_start|>(id 1) message 开始 (im = message)
<|im_end|>(id 2) message 结束
```

预告一下 Part 7 第 4 章：我们要把模型从"文档补全器"变成"问答助手"，靠的就是 chat 格式——用特殊 token 把"谁在说话"标记出来：

```
<|im_start|>user
What is the capital of France?<|im_end|>
<|im_start|>assistant
The capital of France is Paris.<|im_end|>
```

- 模型看到 `<|im_start|>assistant\n` 就会"知道"：轮到我说话了。这就是 SFT 阶段教给它的格式。现在只需要记住：特殊 token 是模型"语言的标点符号"，和文本本身一起编码。

对比 minimind 的 6400 词表：小而精

minimind 用的正是 6400 词表。对比一下主流模型：

模型	词表大小
GPT-2 / GPT-3	~50,000
Llama 2	32,000
minimind	**6,400**

- 为什么 minimind 选 6400？因为它是 ~26M 参数的小模型。embedding 层占用的参数 = `vocab_size` × `hidden_size`，词表每大一倍，embedding 就翻一倍。对大模型 5 万词表无所谓，对 26M 的小模型，6400 是"小而精"的平衡点——英文压缩效果足够，参数占用可控。
- ⚠ 注意：我们的数据是英文莎士比亚。6400 词表对英文很够用；但如果做中文，字符数量更大，通常需要更大的词表（几万）——这是语言特性决定的，不是算法问题。

学完这部分你能...

- 讲清"字符级 / 词级 / subword"三难问题，说透为什么 subword 是平衡点
- 用手推一遍 BPE 算法（初始字符集 → 反复合并最高频相邻对 → 达到目标词表）
- 用 HuggingFace `tokenizers` 训练一个 6400 词表的 BPE，理解 `vocab_size/special_tokens/ByteLevel`
- 对比字符级（65）与 BPE（6400）的压缩率（≈3.5×），明白模型代码为什么不用改
- 说出 `<|im_start|>` / `<|im_end|>` 的作用，读懂 chat 格式

课后练习

<details>

<summary>Q1: 为什么特殊 token（如 <|im_start|>）必须在训练 BPE 时就用 special_tokens 预留？</summary>

A: 因为 BPE 训练器是"从统计里长出来"的——它只产生文本里出现过的片段。如果不用 special_tokens 预留，<|im_start|> 这些词要么被当普通文本合并掉、要么根本不在词表里，模型就没有固定的 id 来表示"user 说话开始"。预留之后，训练器会给它们固定的词表位置（通常是词表前几个），之后 encode/decode 才能稳定使用。

</details>

<details>

<summary>Q2: 把 tokenizer 从字符级换成 BPE 后，Part 6 的 Transformer 模型代码需要改哪些地方？block_size 呢？</summary>

A: 模型代码几乎不用改——它只依赖 vocab_size（65 → 6400）这个数字，nn.Embedding 和 lm_head 的维度会自动跟着变。但通常应该增大 block_size：BPE 序列更短，同样的上下文长度能覆盖更多真实内容；同时因为词表变大、每个 token 信息量变大，通常也需要把模型做大一点（hidden 增大）来承载。

</details>

<details>

<summary>Q3: 为什么 BPE 没有 OOV 问题？"ChatGPT"这种训练时没见过的词怎么编？</summary>

A: 因为 ByteLevel BPE 从 UTF-8 字节出发，单字节（甚至空字节）永远在词表里。没见过的高频组合就拆成更小的子词，再不行就退化成单个字符/字节——总能编码。代价是"越陌生的词，token 越长"，但永远不会编不出来。这正是 subword 相对词级最大的优势。

</details>

课后作业

完成本章后，去 Assignment 7 完成题 1（BPE 编码）：

[Assignment 7](../../assignments/assignment_7/)

下一步

tokenizer 搞定了：文本变成了更短、信息更密的 6400 子词序列。但模型的骨架还是 Part 6 那套——LayerNorm + learned

位置编码。下一步我们开始"换零件"：先换归一化（RMSNorm），再换位置编码（RoPE）。

[02 — 现代组件：RMSNorm 与 RoPE](02_modern_components.md)

02_modern_components

02 — 现代组件：RMSNorm 与 RoPE

> Transformer 的骨架没变，但两个关键零件换成了"现代款"：归一化从 LayerNorm 换成 RMSNorm，位置编码从可学习参数表换成 RoPE。本课把这两个零件从零实现。

学习目标

完成本章后，你将能够：

- 手写 RMSNorm 前向（并解释它砍掉 LayerNorm 的哪两步、为什么安全）
- 推导 RoPE 的旋转矩阵形式与"内积只依赖相对位置"性质
- 实测 naive/PI/NTK/YaRN 四种位置方案的外推行为并解释排序（进阶小节 + 脚本 11/13）

前置知识

• 必须掌握：Part 6 03 章（LayerNorm 的 pre-norm、残差连接——RMSNorm 一节的对照基准）；Part 3 的 BatchNorm（归一化的"列"、训练/推理两态、 γ/β 可学习参数——归一化家族的共同语言）

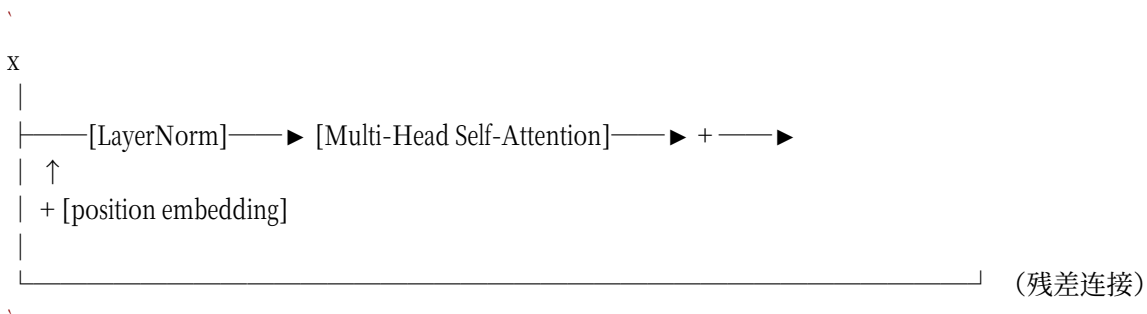
• 建议掌握：Part 6 02 章（位置编码的作用、self-attention 里 q/k/v 怎么用——RoPE 推导要在这上面展开）

• 可选：复数旋转表示的直觉（无也不影响，推导从二维旋转矩阵讲起）

> 重点回看 Part 6 里 LayerNorm 的实现（**weight/bias** 两个可学习参数）——RMSNorm 是它的"瘦身版"。

从 Part 6 结束的地方出发

Part 6 的 Transformer Block 长这样：



两个"零件"这一章要被换掉：

1. LayerNorm $\rightarrow$ RMSNorm：归一化方式
2. learned positional embedding（第二张 embedding 表） $\rightarrow$ RoPE：位置编码方式

为什么换？这一章我们不只讲"怎么换"，更讲"为什么"。先说归一化。

归一化回顾：从 BatchNorm 到 LayerNorm

Part 3 我们实现了 BatchNorm：对一个 batch 的所有样本，按"列"（每个特征）减去均值、除以标准差，然后用可学习的 γ/β 做缩放和平移：

BatchNorm: $y = \gamma \cdot (x - \text{mean_batch}) / \sqrt{\text{var_batch} + \text{eps}} + \beta$

↑ 按 batch 统计 ↑ 可学习缩放 ↑ 可学习平移

Part 6 我们换了 LayerNorm：归一化的对象从"batch 列"换成"单个样本的行"，并且不再需要 running buffer（没有训练/推理两态，直接对所有位置归一化）：

LayerNorm: $y = \gamma \cdot (x - \text{mean_row}) / \sqrt{\text{var_row} + \text{eps}} + \beta$

↑ 按单个样本统计 ↑ 可学习缩放 ↑ 可学习平移

• 注意 LayerNorm 有两个可学习参数： γ （缩放）和 β （平移），还有一个可选的 **bias**。它们让归一化之后的分布"不完全固定"，网络能自己学出合适的分布。

RMSNorm：只算均方根，砍掉均值和平移

公式

RMSNorm（Root Mean Square Normalization, 2019）的核心洞察：均值中心化在 Transformer 里信息量很低，可以砍掉。

先看 RMSNorm 怎么定义。它不做均值中心化，只把每个样本的每一行除以自己的均方根（RMS）：

$\text{RMSNorm}(x) = x / \sqrt{\text{mean}(x^2) + \text{eps}} * \text{weight}$

↑ ↑ ↑

保持原值 只算平方的均值 可学习缩放（不再有 β /bias）

逐项拆解：

- $\text{mean}(x^2)$ ：对每个位置的 hidden 向量，把每个元素的平方取平均（注意：不先减均值）
- 开根号 + eps：得到"激活的均方根"，eps 防除零
- 除以它：把整行的"尺度"归一化到接近 1
- **weight**：可学习的 γ ，逐元素缩放

代码：RMSNorm

对照 minimind 的实现，[02_rmsnorm_rope.py](../scripts/02_rmsnorm_rope.py) 里：

```
python
class RMSNorm(nn.Module):
    """只做均方根归一化，砍掉均值中心化和 bias"""
    def __init__(self, dim, eps=1e-5):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim)) # 只有一个可学习参数  $\gamma$ 
    def forward(self, x):
        rms = torch.sqrt(x.pow(2).mean(-1, keepdim=True) + self.eps) # 均方根
        return self.weight * (x / rms) # 除以尺度，再缩放
```

• ⚠ 注意和 LayerNorm 的三个区别：① 没有减均值；② 没有 β ；③ 没有 bias。**nn.Parameter** 只有 **weight** 一个。

- `mean(-1,keepdim=True)` 是沿最后一维（hidden 维）取平均，`keepdim` 保留维度好做广播。rms 的 shape 是 $(B, T, 1)$ ，和 x 广播相除。

数值例子：RMSNorm vs LayerNorm（手算）

拿一个 1D 向量手算一遍，对比最直观。设 $x = [1, 2, 3]$ ，暂时忽略 `weight` 和 `eps`：

RMSNorm:
 $\text{mean}(x^2) = (1^2 + 2^2 + 3^2)/3 = 14/3 \approx 4.667$
 $\text{rms} = \sqrt{4.667} \approx 2.160$
 $y = x / 2.160 \approx [0.463, 0.926, 1.389]$
验证：输出自己的均方根 = $\sqrt{((0.463^2 + 0.926^2 + 1.389^2)/3)} = 1$ 归一化到 RMS=1

LayerNorm（同样输入）：
 $\text{mean} = (1+2+3)/3 = 2$
 $\text{var} = ((1-2)^2 + (2-2)^2 + (3-2)^2)/3 = 2/3 \approx 0.667$
 $y = (x - 2)/\sqrt{0.667} \approx [-1.225, 0, 1.225]$

- 注意两者的差异：LayerNorm 先减均值，输出必然以 0 为中心（有正有负）；RMSNorm 不做减均值，输出保持原来的正负形状，只是把"尺度"压成 1。所以 RMSNorm 对"激活本身的值"更忠实——这正是"均值中心化信息量低，砍掉它"的直观体现。
- γ 我们的例子为了直观忽略了 `weight` (γ)。真实模型里 γ 初始化为 1，训练中自己学， $y = \gamma \odot (x/\text{rms})$ 逐元素缩放——和 LayerNorm 的 γ 作用相同。

为什么 RMSNorm 更好？

1. 均值中心化在 Transformer 里信息量低
Transformer 的隐藏层经过残差连接，数值分布已经被多次混合，均值这一项几乎不携带有用信息。去掉它，模型几乎不掉点（原文实验里 LayerNorm 换 RMSNorm 性能相当）。
 2. 省掉 β /bias，参数更少、计算更省
LayerNorm 每个 hidden 维要存 γ 和 β 两个参数；RMSNorm 只要 γ 。在 8 层、hidden 512 的模型里，省的参数不多，但计算上少了一次减均值、一次算均值，前向/反向都更快。对超大规模模型，这种"每层省一点"会累积成可观的加速。
 3. 训练更稳定
归一化层的数值只依赖"平方的均值"，函数更平滑、梯度更干净，训练大模型时被证明更稳（这也是 Llama 系列选择它的原因之一）。
- > 一句话：RMSNorm = LayerNorm 去掉"减均值"和"可学习平移"，只保留"除以均方根 + 缩放"。用更少的计算换回相当的性能，是现代 LLM 的默认选择。

对比表

	LayerNorm	RMSNorm
公式	$(x - \mu) / \sqrt{(\sigma^2 + \text{eps})} \cdot \gamma + \beta$	$x / \sqrt{(\text{mean}(x^2) + \text{eps})} \cdot \gamma$
减均值	有	**没有**
可学习参数	$\gamma + \beta$	**只有 γ **
bias	有（可关）	**没有**

	LayerNorm	RMSNorm
计算量	算 mean + var	**只算 $\text{mean}(x^2)$ **
训练稳定性	好	**更好**（小模型也常用）

顺带一提：Query/Key 归一化

minimind 在注意力里还对 q 和 k 各自又加了一层 RMSNorm（q_norm/k_norm，head_dim 维）：

```
`python
xq = self.q_norm(xq) # 每个 head 的 q 再归一化一次
xk = self.k_norm(xk)
`
```

- 这层在注意力的"缩放"之外又加了一重归一化，主要作用是把 q/k 的尺度稳定住（内积对尺度很敏感）。它是近几年的流行做法，不是必需——先知道有这回事，重点还是 block 里那两个大的 RMSNorm。
- △ 我们的教程脚本（[03_gqa_kv_cache.py](../scripts/03_gqa_kv_cache.py)、[05_full_model.py](../scripts/05_full_model.py)）省略了 q_norm/k_norm，以简化实现、聚焦核心概念。它是可选增强，不影响对 GQA/RoPE 的理解。minimind 的某些版本包含它。

RoPE：从"可学习的参数表"到"旋转位置编码"

Part 6 的做法回顾：learned positional embedding

Part 6 里位置信息靠第二张 embedding 表：`nn.Embedding(block_size, n_embd)`，每个位置一个可学习的向量，加到 token embedding 上：

```
`python
tok_emb = token_embedding_table(idx) # (B,T,C) token 本身
pos_emb = position_embedding_table(arange(T)) # (B,T,C) 位置向量
x = tok_emb + pos_emb # 相加注入位置信息
`
```

- △ 它的两个短板：① 要花参数（block_size × hidden 一张表）；② 不能外推——训练时最长只见过 block_size 个位置，推理时序列一超过这个长度，位置编码就"越界"，模型表现崩掉。
- 回想 Part 6 提过的 nanoGPT 用 `nn.Embedding` 位置编码，同样受制于"训练长度 = 最大长度"。

RoPE 的核心想法：位置 = 旋转

RoPE（Rotary Position Embedding，旋转位置编码，2021）的思路完全不同：不学一张位置表，而是把 q 和 k 向量按它们的位置"旋转"一个角度。

为什么"旋转"能编码位置？因为在二维平面上，旋转一个向量不改变它的长度（范数），但会改变它和另一个向量的夹角——而注意力算的是内积，内积恰好只取决于"夹角 × 长度"。

> 两个向量做内积，如果只把其中一个旋转 θ 度、另一个不动，内积会按 $\cos(\text{位置差})$ 的规律变化。这个变化只取决于两个位置之差（相对位置），与绝对位置无关——这正是我们想要的"相对位置感知"。

数学：从旋转矩阵到频率

一个二维向量 (x_0, x_1) 旋转 θ 角，用旋转矩阵表示：

$$\begin{bmatrix} x_0' \\ x_1' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$$

$$x_0' = x_0 \cdot \cos \theta - x_1 \cdot \sin \theta$$

$$x_1' = x_0 \cdot \sin \theta + x_1 \cdot \cos \theta$$

• 等价写法是复数：把 (x_0, x_1) 看成复数 $z = x_0 + i \cdot x_1$ ，旋转就是乘 $e^{i\theta}$ ——旋转位置编码的官方推导就是这么写的。 θ 是"这一维的旋转频率"。

但 hidden 维是几百维，不是 2 维。做法是：把 hidden 维两两分成一组，每一组用不同的旋转频率 θ ，沿维度呈指数变化：

$$\text{freq}[i] = 1 / \theta^{(2i / \text{dim})} \quad i = 0, 1, 2, \dots$$

$$\text{angle} = \text{position} * \text{freq}[i] \quad \text{第 } i \text{ 组在这个 position 上旋转 angle 弧度}$$

• θ (即 RoPE 的 `rope_base`) 通常取 $1e4 \sim 1e6$ ，minimind 取 $1e4$ (与 Llama 系列一致)

• 第 0

组频率最高 (转得快)，后面的组频率指数衰减 (转得慢)——低频慢转、高频快转，和傅里叶分解同理

代码：`precompute_freqs_cis + apply_rotary_pos_emb`

minimind 的实现分两步，`[02_rmsnorm_rope.py](../scripts/02_rmsnorm_rope.py)` 里照抄并加了注释：

第一步：预先算好每个位置的 cos/sin 表 (一次性，缓存为 buffer)

```
python
```

```
def precompute_freqs_cis(dim, end, rope_base=1e4):
```

1) 每个维度对的频率： $1/\theta^{(2i/\text{dim})}$

```
freqs = 1.0 / (rope_base ** (torch.arange(0, dim, 2)[:dim // 2].float() / dim))
```

2) 位置 t 与频率做外积 $\rightarrow$ $(\text{end}, \text{dim}/2)$ 的角度矩阵

```
t = torch.arange(end)
```

```
angles = torch.outer(t, freqs).float()
```

3) 转成 cos/sin，并在最后拼接一份 (配合 `rotate_half` 的"对半翻转"技巧)

```
cos = torch.cat([torch.cos(angles), torch.cos(angles)], dim=-1)
```

```
sin = torch.cat([torch.sin(angles), torch.sin(angles)], dim=-1)
```

```
return cos, sin # 各自 shape (max_pos, dim)
```

• `torch.outer(t, freqs)` 得到 $(\text{end}, \text{dim}/2)$ 的矩阵， $[i, j]$ 就是"位置 i 的第 j 组角度"。拼接成 dim 长是为了下面的 `rotate_half` 技巧 (对半翻转后逐元素相乘)，省一次显式矩阵乘法。

第二步：把 cos/sin 应用 (旋转) 到 q 和 k 上

```
python
```

```
def rotate_half(x):
    """把后一半取负放到前一半 → 等价于复数的  $i \cdot z$ """
    return torch.cat([-x[..., x.shape[-1] // 2:], x[..., : x.shape[-1] // 2]], dim=-1)
```

```
def apply_rotary_pos_emb(q, k, cos, sin):
```

q 在位置 p 上旋转 p 的角度 ($\cos_p, \sin_p$ 是预计算表里第 p 行)

```
q_rot = (q cos) + (rotate_half(q) sin)
k_rot = (k cos) + (rotate_half(k) sin)
return q_rot, k_rot
```

- $q \cos + \text{rotate_half}(q) \sin$ 这行把"复数乘 $e^{i\theta}$ "翻译成了实数运算： $\text{rotate_half}(q)$ 恰好实现了 $i \cdot q$ (后一半取负放到前一半)。这样在 q/k 上各乘一下，就等价于让它们各自旋转。
- 旋转是逐元素的：不需要学参数，只需要查表。位置信息以"旋转角度"的形式被揉进了 q 和 k。

> 脚本实现对照：上面用实数版 ($\cos/\sin + \text{rotate_half}$) 讲原理，更直观。实际脚本 [02_rmsnorm_rope.py](./scripts/02_rmsnorm_rope.py) 和 [05_full_model.py](./scripts/05_full_model.py) 用复数版实现——把 (x_0, x_1) 看成复数 $z = x_0 + i \cdot x_1$ ，用 $\text{torch.polar}(1, \text{angle})$ 构造旋转因子 $e^{i\theta}$ ，再用 $\text{torch.view_as_complex} / \text{torch.view_as_real}$ 做复数乘法。两者数学上完全等价 ($q \cdot \cos + \text{rotate_half}(q) \cdot \sin == \text{view_as_real}(\text{view_as_complex}(q) * e^{i\theta})$)，复数版代码更简洁，但需要了解 $\text{torch.view_as_complex}$ 等 API。作业题 3 (RoPE) 的测试对两种实现都接受。

第三步：在 attention 里使用

```
xq, xk 算出来后：
1. 取当前位置范围的 cos/sin：cos[pos:pos+seq], sin[pos:pos+seq]
2. xq, xk = apply_rotary_pos_emb(xq, xk, cos, sin) # 旋转注入位置
3. 之后照常算内积 (xq @ xk^T)
```

为什么 RoPE 更好：相对位置、可外推、零参数

1. 绝对位置不影响内积，相对位置决定内积

旋转是正交变换： $\| \text{旋转后的向量} \| = \| \text{原向量} \|$ 。q、k 各自旋转后，内积变成 $q \cdot k \cdot \cos(\text{角度差})$ ——绝对位置完全不影响（都旋转不改变夹角差的部分...严格说内积依赖角度差），这正是位置编码想要的"相对位置感知"。相比 learned PE 要硬记位置对，RoPE 直接把相对距离编码进了内积。

2. 可外推 (extrapolation)

训练时 max position 4096，推理时想要 8192？RoPE 的 $\cos/\sin$

表是公式生成的， $\text{precompute_freqs_cis}(\text{end}=8192)$ 就能算出来——不需要重新训练。learned PE 没有这个能力（表就是参数，没见过就是没见过）。

△ 严格说"直接外推"超过训练长度太多，模型精度还是会掉（长上下文的高频维度分布变了）。工业界用 YaRN / NTK scaling 这类技巧缓解，minimind 也支持 ($\text{inference_rope_scaling}$)。我们教程已经做了 scaling 实测：`[scripts/11_rope_scaling.py](./scripts/11_rope_scaling.py)`

用同一模型对比四种方案 (naive/PI/NTK/YaRN) "训练 128 → 推理 256"的困惑度——YaRN 的温度因子 $\sqrt{1/t} = 0.1 \cdot \ln(s) + 1$ (论文/HF 官方做法： $\sqrt{1/t}$ 同时乘 q、k，等价 $\text{logit} \times 1/t$ ；本课脚本简化为只乘

q) 是面试加分点；`[scripts/13_long_context_eval.py](./scripts/13_long_context_eval.py)` 再用迷你 RULER 的 KV 检索任务量"外推后还记得住吗"，两份实测数字见第 5 章「进阶实验」。

3. 零参数

RoPE 不引入任何可学习参数。对比 learned PE 那张 $\text{block_size} \times \text{hidden}$ 的 embedding 表，RoPE 只占一小块 $\cos/\sin$ 缓存（buffer，不算参数）。参数省了，还顺带解决了外推。

数值例子：旋转 2D 向量，看"相对位置决定内积"

用最简单的 2D 向量感受一下"旋转为什么能编码相对位置"。设每个位置的旋转频率 $\theta = 0.5$ rad/位置，两个单位向量 $q = k = [1, 0]$ （长度都是 1，范数不变）。

位置 2 的 q ：旋转 $2 \times 0.5 = 1.0$ rad $\rightarrow q_2 = [\cos 1.0, \sin 1.0]$

位置 2 的 k ：旋转 $2 \times 0.5 = 1.0$ rad $\rightarrow k_2 = [\cos 1.0, \sin 1.0]$

内积 $q_2 \cdot k_2 = \cos(0) = 1.0 \leftarrow$ 相同位置，完全对齐

位置 2 的 q 、位置 3 的 k ：

内积 $= \cos((3-2) \times 0.5) = \cos(0.5) \approx 0.878 \leftarrow$ 相邻，轻微错开

位置 5 的 q 、位置 6 的 k ：

内积 $= \cos((6-5) \times 0.5) = \cos(0.5) \approx 0.878 \leftarrow$ 同样是"相差 1"，结果一样！

位置 2 的 q 、位置 8 的 k ：

内积 $= \cos((8-2) \times 0.5) = \cos(3.0) \approx -0.99 \leftarrow$ 隔得远，几乎反向

- 关键观察：第 2、3 组"位置差都是 1"，内积都是 0.878——虽然它们的绝对位置不同（2/3 和 5/6），结果一模一样。内积只取决于位置差，与绝对位置无关。这就是"旋转正交、范数不变"带来的性质。
- 把这里的 $\theta = 0.5$ 换成真实 RoPE 的多组频率，同一套直觉依然成立：相邻 token 注意力分数高，相隔越远分数越低，且不依赖绝对位置。

把 RoPE 装进 attention

在完整注意力里，RoPE 只改两个位置： q/k

旋转、然后照常算内积。[02_rmsnorm_rope.py](../scripts/02_rmsnorm_rope.py) 里：

```
python
```

1. 预计算好 $\cos/\sin$ 表（模型初始化时算一次，存成 buffer）

```
cos, sin = precompute_freqs_cis(dim=head_dim, end=max_position_embeddings)
```

```
model.register_buffer('cos', cos, persistent=False)
```

```
model.register_buffer('sin', sin, persistent=False)
```

2. forward 里， q/k 算出来后取当前位置的 $\cos/\sin$ 并旋转

```
q = q_proj(x).view(B, T, n_heads, head_dim) # (B,T,8,hd)
```

```
k = k_proj(x).view(B, T, n_heads, head_dim)
```

```
cos_t = self.cos[:T].unsqueeze(0).unsqueeze(0) # (1,1,T,hd) 取前 T 个位置
```

```
sin_t = self.sin[:T].unsqueeze(0).unsqueeze(0)
```

```
q, k = apply_rotary_pos_emb(q, k, cos_t, sin_t) # 旋转注入位置
```

3. 之后照常： $\text{scores} = (q @ k^T) / \sqrt{\text{head_dim}} \rightarrow \text{softmax} \rightarrow @v$

- ⚠ 两个容易踩的坑：① `cos/sin` 只取 `[:T]`——位置从 0 数起，正好和 token 下标对齐；② 做生成带 KV Cache 时从 `start_pos` 偏移取（否则新 token 的位置算错）。第 3 章讲 KV Cache 时会再碰这个坑。

对比表

	learned PE (Part 6)	RoPE (Part 7)
形式	一张可学习 embedding 表，加到 token 上	旋转 q/k，公式生成
参数	<code>'block_size × hidden'</code>	<code>**0**</code>
依赖训练长度	是，超过就崩	<code>**否，可外推**</code>
相对位置	硬记（要大量数据学）	<code>**结构内建（旋转角差）**</code>
绝对位置	显式编码	<code>**不影响内积**</code>

权重绑定 (tie_word_embeddings)

最后一个小零件：把 token embedding 和最后的输出层 (lm_head) 共享同一个权重。

`embed_tokens`: vocab(6400) → hidden(512) 输入侧：token → 向量
`lm_head`: hidden(512) → vocab(6400) 输出侧：向量 → token 分数

如果两个权重一样，参数直接从"嵌入"省成"一份"

```
python
self.lm_head = nn.Linear(config.hidden_size, config.vocab_size, bias=False)
if self.tie_word_embeddings:
    self.model.embed_tokens.weight = self.lm_head.weight # 指向同一份权重！
```

- 这个技巧叫 weight tying (权重绑定)：embedding 层学到的"每个 token 的向量表示"，反过来也能当"预测每个 token 的分数向量"用。省掉 `vocab × hidden` 一整块参数——对我们 26M 的小模型，这一省就是 $6400 \times 512 \approx 3.3\text{M}$ ，约 12%。
- ⚠ 为什么可以共享？直觉：一个 token 越"容易被预测"，说明它的表示越有区分度，用同一个向量既能"表示它"又能"给它打分"是自治的。GPT-2 之后的很多模型默认开启。
- 用了绑定之后，`embed_tokens.weight` 和 `lm_head.weight` 是同一份参数（内存共享），PyTorch 里梯度会自动累计到同一个 `Parameter` 上，不需要额外处理。

学完本部分你能...

- 写出 RMSNorm，讲清它与 LayerNorm 的三个区别（无减均值、无 β 、无 bias）及为什么更优
- 画出 `q_norm/k_norm` 在注意力里的位置，知道它是可选增强
- 用复数/旋转矩阵解释 RoPE："旋转 → 内积依赖角度差 → 相对位置"
- 手写 `precompute_freqs_cis + apply_rotary_pos_emb`，在 attention 里注入位置
- 对比 learned PE 与 RoPE（参数、外推、相对位置），说清"为什么 RoPE 是默认"
- 说出 tie_word_embeddings 省了哪块参数 (`vocab × hidden`)

课后练习

<details>

<summary>Q1: 为什么 LayerNorm 去掉均值中心化后性能几乎不掉？</summary>

A: 均值中心化的作用是让激活"以 0 为中心", 但 Transformer

里每一层都被残差连接叠加, 激活的均值分布已经被混合得很有规律, 均值项携带的信息量很低。而且 RMSNorm 保留了"除以均方根"和"可学习缩放 γ "这两个真正起作用的归一化因子。实验证明, 对激活做不做均值中心化对最终表现影响很小, 去掉反而省计算。

</details>

<details>

<summary>Q2: 为什么 RoPE 能让"绝对位置不影响内积"? 这对语言建模有什么用? </summary>

A: 旋转是正交变换, 范数不变。q、k 各自旋转后, 内积 $\langle R(q), R(k) \rangle$ 只与它们的角度差有关——而角度差正是由位置差决定的。所以内积(注意力权重)只反映相对距离: 相邻 token 得分高、相隔远的得分低, 且与"这发生在序列的第 5 位还是第 500 位"无关。这对语言建模正合适: 单词的意义更多取决于它和上下文的相对位置, 而不是它在整个语料里的绝对序号。

</details>

<details>

<summary>Q3: 权重绑定为什么能省参数? 它有没有副作用? </summary>

A: 输入侧把 token 变成向量、输出侧把向量变成 token 分数, 两边的形状都是 $\text{vocab} \times \text{hidden}$, 本质是同一类"token $\leftrightarrow$ 向量"映射。让它们共享同一份权重, 省掉 $\text{vocab} \times \text{hidden}$ 一整块(我们的小模型约 12%)。副作用通常是极小的(输出和输入侧的任务不完全一样, 共享算一点点"参数共享正则化"), 实践中往往还能略微提升小模型效果, 所以 GPT-2、Llama 这类模型默认开启。

</details>

课后作业

完成本章后, 去 Assignment 7 完成题 2 (RMSNorm) 和题 3 (RoPE) :

[Assignment 7](../../assignments/assignment_7/)

下一步

归一化和位置编码都换成了"现代款"。但注意力的内部还有一个大问题没解决: 8 个 Q 头各自配了 8 套独立的 K/V, 太费内存了。下一步我们把注意力升级成 GQA、加上 KV Cache, 再把 FFN 从 ReLU 换成 SwiGLU, 并看一眼 MoE。

[03 — GQA 与 FFN : SwiGLU、KV Cache、MoE](03_gqa_and_ffn.md)

03_gqa_and_ffn

03 — GQA 与 FFN : KV Cache、SwiGLU、MoE

> 注意力和前馈网络是现代 LLM 里最"吃资源"的两个部件。本课把 MHA 升级成 GQA、加上 KV Cache, 把 ReLU FFN 换成 SwiGLU, 最后看看"专家" (MoE) 是什么。

前置知识

- 必须掌握：[Part 6 02 章](../Part6_transformer/tutorial/02_attention_from_scratch.md) (Multi-Head Self-Attention——GQA/KV Cache 全在它之上)
- 建议掌握：[Part 6 03 章](../Part6_transformer/tutorial/03_transformer_block.md) (FeedForward、残差连接——SwiGLU/MoE 的对照面)
- 可选：[Part 2](../Part2_mlp/tutorial/02_mlp_architecture.md)/[Part 5](../Part5_wavenet/tutorial/02_wavenet_architecture.md) 的非线性激活函数与负对数似然

> 重点回看 Part 6 的 4D batched multi-head (把"头"塞进 batch 维) ——GQA 就是在它基础上"砍掉一半 K/V"。

从 Part 6 结束的地方出发

Part 6 我们实现的 attention 是 MHA (Multi-Head Attention) : `n_head` 个头, 每个头都有自己独立的 Q、K、V 线性层。8 个头就是 8 套 Q/K/V。

这一章要回答一个问题: K/V 真的需要每个头都来一套吗?

GQA: K/V 是显存瓶颈

回顾 MHA: 每头一套 K/V

MHA (8 头) :

Q 头 0,1,...,7 —— 各要一套 (Q 必须每头独立, 负责"关注什么")

K 头 0,1,...,7 —— 每头一套

V 头 0,1,...,7 —— 每头一套

- 为什么 Q 必须每头独立? 因为每个 Q 头代表一种"注意力视角" (有的看语法、有的看指代、有的看语义), 它们必须不一样才能各司其职。
- $\triangle$ 但 K/V 是"被查询的内容", 8 个头查的内容其实高度重合。为 8 个头各存一套 K/V, 很浪费。

关键问题: 序列越长, K/V 越占显存

推理时, 模型要缓存所有已生成的 K/V 向量 (这就是下文的 KV Cache), 用于计算新 token 的注意力。缓存大小正比于:

KV 缓存大小 $\approx n_{\text{layers}} \times n_{\text{kv_heads}} \times \text{seq_len} \times \text{head_dim}$

- 序列每长一倍, K/V 缓存大一倍; 模型层数越多、越大, K/V 缓存越爆炸。长上下文的瓶颈不在"计算", 而在"K/V 的显存"。GQA 正是为压这个指标而生的。

两端的尝试: MHA 与 MQA

MHA (Part 6) : 每头独立 K/V

8 套 K/V, 质量最高, 但 K/V 缓存最大

MQA (Multi-Query Attention) : 所有 Q 头共享同一组 K/V

1 套 K/V, K/V 缓存最小, 参数最少

但"一刀切"太狠, 不同头被迫用同一份 K/V, 质量下降

- MQA : 8 个 Q 头共享 1 套 K/V。缓存压到 1/8, 但表达能力受损。

GQA : 分组共享, 折中

从 Part 6 的 MHA 到 GQA, 代码只改 3 处 (左侧 [Part 6 脚本

05](../Part6_transformer/scripts/05_multihead_feedforward.py), 右侧 [Part 7 脚本

05](../scripts/05_full_model.py)) :

```
`diff
class Attention:
def __init__(self, n_embd, n_head, ...):
self.n_heads = n_head
+ self.n_kv_heads = n_kv_heads # ① 新增: KV 头数 (如 8 头里只留 4 组 K/V)
+ self.n_rep = self.n_heads // self.n_kv_heads
  • self.key = nn.Linear(n_embd, n_embd) # ② K/V 投影输出维从 n_embd 缩到
  • self.value = nn.Linear(n_embd, n_embd) # n_kv_heads * head_dim
+ self.wk = nn.Linear(n_embd, self.n_kv_heads * self.head_dim, bias=False)
+ self.wv = nn.Linear(n_embd, self.n_kv_heads * self.head_dim, bias=False)

def forward(self, x):
...
  • k = self.key(x).view(B, T, self.n_heads, head_dim) # ③ 用之前把 K/V
  • v = self.value(x).view(B, T, self.n_heads, head_dim) # 复制回 Q 的头数
+ k = self.wk(x).view(B, T, self.n_kv_heads, head_dim)
+ v = self.wv(x).view(B, T, self.n_kv_heads, head_dim)
+ k, v = repeat_kv(k, self.n_rep), repeat_kv(v, self.n_rep)
```

其余 (Q 投影、softmax、加权求和) 一行都不用动——这就是"换零件不改骨架"。

GQA (Grouped-Query Attention, 2023) 把 Q 头分组, 每组共享一套 K/V :

GQA (8 Q 头 / 4 KV 头, 2:1 分组) :

Q 头 0,1 → K/V 组 0 ← 头 0 和 1 共用 K/V 组 0

Q 头 2,3 → K/V 组 1

Q 头 4,5 → K/V 组 2

Q 头 6,7 → K/V 组 3

- minimind 用的正是 8 Q 头 / 4 KV 头 = 2:1。每个 KV 头服务 2 个 Q 头。K/V 缓存直接减半 (4 套 vs 8 套), 而质量损失远小于 MQA——用"分组"做平滑的折中。

- GQA 还是"参数共享"的另一种形式: KV 的线性层只有 $4 \times \text{head_dim} \times \text{hidden}$, 而 MHA 要 $8 \times \text{head_dim} \times \text{hidden}$, 省了一半 K/V 参数。

repeat_kv：把 K/V 广播回每个 Q 头

训练/推理时，Q 是 8 个头，K/V 只有 4 组。要把 K/V 复制成 8 份才能和 Q 做矩阵乘法。这就是 repeat_kv：

```
python
def repeat_kv(x, n_rep):
    """x: (B, T, n_kv_heads, head_dim) → 复制 n_rep 份 → (B, T, n_kv_heads*n_rep, head_dim)"""
    bs, slen, num_kv_heads, head_dim = x.shape
    if n_rep == 1:
        return x
    return (x[:, :, :, None, :] # (B,T,nh,1,hd) 扩一维
    .expand(bs, slen, num_kv_heads, n_rep, head_dim)
    .reshape(bs, slen, num_kv_heads * n_rep, head_dim))
```

• $n_rep = n_heads // n_kv_heads = 8 // 4 = 2$ 。expand 是"逻辑复制"（不拷贝内存），reshape 把复制的 2 份摊开。注意 KV 线性层的输出头数是 n_kv_heads (4)，不是 n_heads (8)——这是 GQA 和 MHA 在代码上最直接的差别。

[03_gqa_kv_cache.py](../scripts/03_gqa_kv_cache.py) 里 GQA 的完整 forward：

```
python
q = self.q_proj(x).view(B, T, n_heads, head_dim) # 8 头
k = self.k_proj(x).view(B, T, n_kv_heads, head_dim) # 4 组 ← 注意！
v = self.v_proj(x).view(B, T, n_kv_heads, head_dim) # 4 组
```

应用 RoPE（上章）

q, k = apply_rotary_pos_emb(q, k, cos, sin)

GQA 关键：把 K/V 广播回 8 份，然后转成 (B, n_heads, T, head_dim)

```
k = repeat_kv(k, n_rep).transpose(1, 2) # (B,8,T,hd)
v = repeat_kv(v, n_rep).transpose(1, 2)
scores = (q.transpose(1, 2) @ k.transpose(-2, -1)) / math.sqrt(head_dim)
```

• ⚠ 对比 Part 6 的 MHA：那里 k_proj 输出 n_heads 份、不用 repeat_kv。GQA 只改了两处——KV 的投影头数和多一次 repeat_kv。数学上 GQA 训练结果与"等参数量 MHA"接近，但推理 KV 缓存小一半。

GQA 到底省了多少：参数与 KV 缓存（以 hidden=512 / 8 头 / 8 层为例）

	KV 头数	KV 线性层参数（每层）	KV 缓存/token（每层）	4096 token 全 8 层缓存（fp16）
MHA	8	$2 \times (512 \times 512) \approx **524K**$	$2 \times 8 \times 64 = 1024$	$\approx **67 MB**$
GQA (minimind) **	**4	$2 \times (512 \times 256) \approx **262K**$	$2 \times 4 \times 64 = 512$	$\approx **33 MB**$
MQA	1	$2 \times (512 \times 64) \approx **65K**$	$2 \times 1 \times 64 = 128$	$\approx **8 MB**$

• 读这张表：GQA 相对 MHA 把 KV 参数和 KV 缓存都减半（262K vs 524K、33MB vs 67MB），而质量损失远小于 MQA。KV 缓存随 seq_len 线性增长，序列越长、层数越多，省得越多——这就是大模型长上下文推理几乎都用 GQA 的原因。

- 注意：Q 的投影完全不受影响（还是 8 头、 $8 \times 64 \times 512$ ）。GQA 只"砍 K/V"，不碰 Q——因为"多视角查询"的能力全靠 Q 头。

KV Cache：生成时只算最后一个 token

推理生成（自回归）时，每次只产生一个新 token，然后拿它拼到序列尾部再跑一遍整个 Transformer——这样做非常浪费：前面所有 token 的 K/V 每次都要重算。

KV Cache 的洞察：第 t 步算出的 K/V，在第 $t+1$ 、 $t+2$... 步里完全不变（它们只依赖前面的 token，而前面的 token 不变）。所以：

朴素生成（每个新 token 重跑整个序列）：

step 1: 重算 token 0~0 的 K/V → 输出 token 1

step 2: 重算 token 0~1 的 K/V → 输出 token 2 ← 0~1 的 K/V 重复算了！

step 3: 重算 token 0~2 的 K/V → 输出 token 3 ← 又重复！

带 KV Cache：

step 1: 算 token 0 的 K/V，缓存起来 → 输出 token 1

step 2: 只算 token 1 的 K/V，和缓存拼接 → 输出 token 2

step 3: 只算 token 2 的 K/V，和缓存拼接 → 输出 token 3

- 关键点：注意力里对 token t 而言，K/V 只来自它之前的 token（因果遮罩）。新 token 的 K/V 只由"输入序列"决定，与"之后生成了什么"无关，所以可以缓存、拼接。

- 复杂度对比：朴素生成每步是 $O(T^2)$ （重算全序列），带 KV Cache 每步是 $O(T)$ （只算最后一个 token 的注意力）。生成 N 个 token，从 $O(N^2 \cdot L)$ 降到 $O(N \cdot L)$ ——长文本生成的加速是数量级的。

代码里 KV Cache 就是"拼接 + 存储"：

```
python
```

推理时把历史 K/V 拼接起来（generation 时传入 past_key_value）

```
if past_key_value is not None:
```

```
k = torch.cat([past_key_value[0], k], dim=1) # 历史 K + 新 K
```

```
v = torch.cat([past_key_value[1], v], dim=1) # 历史 V + 新 V
```

只对最后一个位置算注意力（生成时只需最后一行的分数）

```
scores = (q[:, -1:, :, :] @ k.transpose(-2, -1)) / math.sqrt(head_dim)
```

- ⚠ 有了 KV Cache 后，RoPE 的角度要接着之前的位置算（start_pos 偏移），不能从头数——这是"带缓存 + RoPE"最容易踩的坑。

把 GQA + RoPE + KV Cache 组装进一个 Attention

前面是"零件"，这里把它们装成一个完整的 Attention 模块（对照 minimind 的 Attention 类）：

```
python
```

```
class Attention(nn.Module):
```

```
def __init__(self, hidden, n_heads, n_kv_heads):
```

```

super().__init__()
self.n_rep = n_heads // n_kv_heads # 8//4 = 2
self.head_dim = hidden // n_heads
self.q_proj = nn.Linear(hidden, n_heads * self.head_dim, bias=False) # 8 头
self.k_proj = nn.Linear(hidden, n_kv_heads * self.head_dim, bias=False) # 4 组
self.v_proj = nn.Linear(hidden, n_kv_heads * self.head_dim, bias=False) # 4 组
self.o_proj = nn.Linear(n_heads * self.head_dim, hidden, bias=False)

def forward(self, x, cos, sin, past_key_value=None):
    bsz, seq_len, _ = x.shape
    start_pos = past_key_value[0].shape[1] if past_key_value else 0

    q = self.q_proj(x).view(bsz, seq_len, self.n_heads, self.head_dim)
    k = self.k_proj(x).view(bsz, seq_len, self.n_kv_heads, self.head_dim)
    v = self.v_proj(x).view(bsz, seq_len, self.n_kv_heads, self.head_dim)

```

① RoPE：只旋转本段（从 start_pos 接着算）

```

q, k = apply_rotary_pos_emb(q, k, cos[start_pos:start_pos+seq_len],
sin[start_pos:start_pos+seq_len])

```

② KV Cache：拼接历史 K/V（只在推理生成时走）

```

if past_key_value is not None:
    k = torch.cat([past_key_value[0], k], dim=1) # 历史 K + 新 K
    v = torch.cat([past_key_value[1], v], dim=1)
    past_kv = (k, v)

```

③ GQA：把 K/V 广播回 8 份，转成 (B, n_heads, T, head_dim)

```

k = repeat_kv(k, self.n_rep).transpose(1, 2)
v = repeat_kv(v, self.n_rep).transpose(1, 2)
q = q.transpose(1, 2)

```

④ 注意力：生成时 q 只取最后一行，且无需再遮罩（缓存里全是历史）

```

q = q[:, -1:] if past_key_value is not None else q
scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)
if past_key_value is None: # 只有训练/预填充才需要因果遮罩
    scores = scores.masked_fill(torch.triu(torch.ones_like(scores), diagonal=1), float('-inf'))
attn = F.softmax(scores, dim=-1)
out = (attn @ v).transpose(1, 2).reshape(bsz, seq_len, -1)
return self.o_proj(out), past_kv

```

四个步骤的职责非常清晰：① RoPE 给位置，② KV Cache 给历史，③ GQA 广播 KV，④ 算注意力。**start_pos** 是“带缓存 + RoPE”的关键——新 token 的角度要从 **start_pos** 接着转，不能从 0 重新数。

• △ 训练和推理走同一份代码：训练时 **past_key_value=None**（②跳过、④算全序列、带因果遮罩），推理时传入缓存（②拼接、④只算最后一行、且无需再遮罩——缓存里全是当前 token 之前的历史）。这正是现代 LLM 推理加速的全部秘密。

Flash Attention：让 GPU 更快

- > 这些优化本质都是 GPU 内核层面的活（shared memory tiling、fused kernel）——
- > 想亲手写一遍的话，见 [Part 9 CUDA 内核](../Part9_cuda_kernels/tutorial/02_matmul_optimization.md)：
- > 其中 02 章手写的 SMEM tiling 与 Flash Attention 共享同一套核心思想。

Part 6 提过 PyTorch 2.0 的 `F.scaled_dot_product_attention`（SDPA），minimind 默认也用它：

```
`python
if self.flash and (seq_len > 1):
    output = F.scaled_dot_product_attention(
        xq, xk, xv, is_causal=True) # 自带因果遮罩，又快又省显存
else:
    scores = (xq @ xk.transpose(-2, -1)) / math.sqrt(head_dim)
```

... 手动加因果遮罩 ...

,

- Flash Attention 的核心里面没新数学：只是按块（tile）计算、不落整张 attention 矩阵，把对显存的读写从 $O(T^2)$ 降到 $O(T)$ 。结果和普通 attention 数值上几乎一致，只是更快、更省内存。我们用 `F.scaled_dot_product_attention` 一行拿到底。

- > 这些优化本质都是 GPU 内核层面的活（shared memory tiling、fused kernel）——
- > 想亲手写一遍的话，见 [Part 9 CUDA 内核](../Part9_cuda_kernels/tutorial/02_matmul_optimization.md)：
- > 其中 02 章手写的 SMEM tiling 与 Flash Attention 共享同一套核心思想。

SwiGLU：把 FFN 的非线性换掉

回顾 Part 6 的 ReLU FFN

Part 6 的 FeedForward 是：

```
`python
Linear(hidden → 4 · hidden) → ReLU → Linear(4 · hidden → hidden)
```

"开大的空间思考，再压回去"。ReLU 的问题：在 0 处有个尖角（不可导），负半轴直接归零——信息一旦为负就"死了"，且梯度容易在负区间为 0。

SwiGLU：三投影 + 自适应门控

SwiGLU（2020）是 GLU（门控线性单元）家族的一员。GLU 的想法：用另一个投影当作"门"，决定主投影保留多少：

,

```
SwiGLU: FFN(x) = down_proj( silu( gate_proj(x) ) * up_proj(x) )
↑ 门（开关）  ↑ 内容
```

,

拆开看，FFN 变成三个投影（gate/up/down）：

```
`python
```

```

class FeedForward(nn.Module):
def __init__(self, hidden, intermediate):
super().__init__()
self.gate_proj = nn.Linear(hidden, intermediate, bias=False) # 门
self.up_proj = nn.Linear(hidden, intermediate, bias=False) # 内容
self.down_proj = nn.Linear(intermediate, hidden, bias=False) # 压回

def forward(self, x):
return self.down_proj(F.silu(self.gate_proj(x)) * self.up_proj(x))

```

- `gate_proj(x)` 过 `silu`（即 Swish，PyTorch 里叫 `F.silu`）后变成 0~1 之间的"软开关"： $\text{silu}(z) = z \cdot \sigma(z)$ ，在 0 处平滑（不是尖角），负区间不会完全死掉（保留很小但不为 0 的梯度）
- `up_proj(x)` 是"内容"
- 两者逐元素相乘 = "门控内容"：门想放多少就放多少
- `down_proj` 把结果压回 hidden

为什么 SwiGLU 更好？

1. 平滑、梯度干净：silu 处处可导、负区间梯度不为 0，比 ReLU 的"尖角 + 归零"更好优化，小模型上往往更稳。
2. 自适应门控：ReLU 是"硬开关"（<0 一律关），SwiGLU 是"软开关"（每个维度有独立的 0~1 门，由数据学出来）——表达力更强。
3. 效果更好：论文和 Llama 系列证明，同参数量下 SwiGLU 优于 ReLU FFN（Llama 2 的 FFN 就是这个结构）。

> △ 代价：从 2 个投影变成 3 个，中间维度却从经典的 $4 \times$ 缩到 minimind 的 $\sim 3.2 \times (\text{ceil}(\text{hidden} \cdot \pi / 64) \cdot 64)$ ，参数总量和 ReLU FFN 差不多——用"更宽但更高效的结构"换性能。

> $\text{ceil}(\text{hidden} \cdot \pi / 64) \cdot 64$ 这个公式：经典 ReLU FFN 中间维度是 $4 \times \text{hidden}$ ；SwiGLU 多一个投影，为了控制总参数量，minimind 把中间维度缩到约 $3.14 \times \text{hidden}$ （ $\pi \approx 3.14$ ），再向上对齐到 64 的倍数（GPU tensor core 对齐友好，64 是常见的 tile 大小）。脚本 [04_swiglu_ffn_moe.py](../scripts/04_swiglu_ffn_moe.py) 用 `int((math.pi * hidden / 64) + 0.5) * 64` 实现（四舍五入版，效果等价）。

数值例子：silu vs ReLU（手算）

$\text{silu}(z) = z \cdot \sigma(z)$ ， σ 是 sigmoid。拿几个值对比 ReLU：

z	ReLU(z)	silu(z)	说明
-3	0	-0.14	ReLU 直接关死；silu 保留一点"负的微量"
-1	0	-0.27	同上，梯度仍不为 0
0	0	0	平滑经过（ReLU 是尖角）
1	1	0.73	silu 略"收一点"
3	3	2.86	大正值接近线性

- 关键差别在负半轴：ReLU 对任何负数都输出 0（信息"死"了、梯度为 0）；silu 对负数输出很小的负值，梯度仍然存在——优化器能"告诉"这个维度该往哪调，而不是断了信号。门控场景里，这个"软开关"远比"硬截断"平滑。

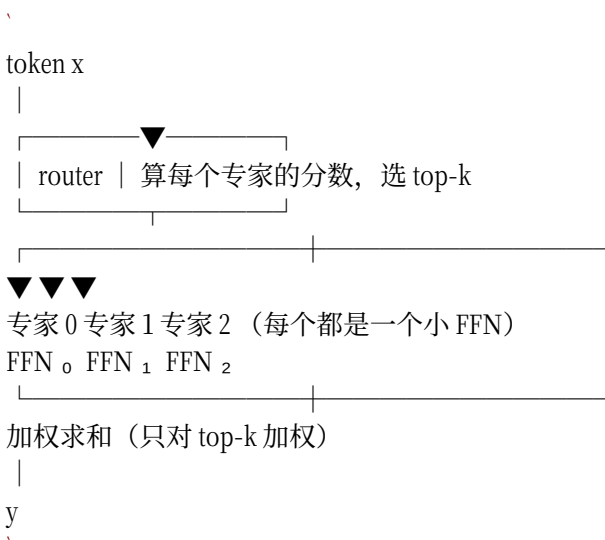
对比表

	Part 6 ReLU FFN	Part 7 SwiGLU
结构	2 投影	**3 投影 (gate/up/down) **
激活	ReLU	**silu (平滑) **
门控	无 (硬截断)	**软门控 (逐维 0~1) **
中间维度	4×	~3.2×
表达力/效果	基线	**更好**

MoE：把 FFN 换成一群"专家"

概念：多个 FFN + 路由器

MoE (Mixture of Experts, 混合专家) 的思路：不只有一个 FFN，而是训练很多个 FFN (专家) + 一个路由器。每个 token 输入时，路由器先给它打个分，只把它路由到最合适的 top-k 个专家去处理：



- 稀疏性：每个 token 只激活 top-k 个专家（如 4 选 1），其它专家“睡觉”。模型参数量很大（一堆专家），但每个 token 的实际计算量很小——“参数多、算力省”。
- 一个直觉：MoE = 按 token 内容“分工”。代码片段走“写代码专家”，散文走“写作专家”。路由器学会这个分工。

代码：top-k 路由

minimind 的 MoE FFN ([04_swiglu_ffn_moe.py](../scripts/04_swiglu_ffn_moe.py), minimind 用 4 专家 / top-1) :

```
python
class MoE(nn.Module):
def __init__(self, hidden, num_experts=4, top_k=1):
super().__init__()
self.gate = nn.Linear(hidden, num_experts, bias=False) # 路由器
```

```

self.experts = nn.ModuleList([FFN(hidden) for _ in range(num_experts)])

def forward(self, x):
    scores = F.softmax(self.gate(x), dim=-1) # 每个专家一个分数
    topk_w, topk_idx = torch.topk(scores, self.top_k, dim=-1) # 取 top-k
    topk_w = topk_w / topk_w.sum(-1, keepdim=True) # top-k 内归一化
    out = torch.zeros_like(x)
    for i, expert in enumerate(self.experts):
        mask = (topk_idx == i) # 哪些 token 选了专家 i
        if mask.any():
            out[mask] += topk_w[mask] * expert(x[mask]) # 加权
    return out

```

• △ 负载均衡 (load balance) : 如果路由器"偏爱"某个专家, 其它专家就废了。所以要加 auxiliary loss (辅助损失) : 惩罚"某个专家被选得过多"。minimind 用 `aux_loss = (load_scores.mean()).sum() num_experts * coef` (load 是每个专家的平均被选次数, scores.mean 是平均得分, 两者乘积大 = 负载不均), 把它加到总损失上。这是 MoE 工程里必做的一步。

minimind 的 MoE 是可选项

- minimind 的默认 `MiniMindConfig(use_moe=False)` 是 Dense (稠密) 模型; 把 `use_moe=True` 就切换成 MoE 版 (4 专家 / top-1)。
- 对我们 26M 的小模型, MoE 属于"锦上添花": 理解概念为主, 训练脚本默认不开 MoE。等你有 GPU 想复现 minimind-MoE (145M) 再开。

学完本部分你能...

- 讲清"为什么 K/V 是显存瓶颈" ($KV \text{ 缓存} \propto n_{kv_heads} \times seq_len$)
- 对比 MHA / MQA / GQA, 说出 minimind 用 8 Q 头 / 4 KV 头 (2:1 分组)
- 手写 `repeat_kv`, 指出 GQA 与 MHA 在代码上的差别 (KV 投影头数)
- 实现 KV Cache : 生成时只算最后 token、复用历史 K/V, 理解 $O(T^2) \rightarrow O(T)$ 的加速
- 手写 SwiGLU (gate/up/down 三投影), 对比 ReLU FFN
- 讲清 MoE 的路由、top-k、负载均衡损失, 知道它是 minimind 的可选项

课后练习

<details>

<summary>Q1: 为什么 GQA 能省显存却不怎么掉质量? "分组"到底在省什么? </summary>

A: 省的是 K/V 的"套数" (KV 头数) : MHA 每头一套, GQA 把 Q 头分组、每组共享一套。KV 缓存正比于 KV 头数, 所以 8→4 就减半。质量损失小的原因是: K/V 表达的是"被查询的内容", 不同 Q 头对内容的需求高度重合, 分组合并只是去掉冗余; 而 Q 头仍然各自独立, 保留了"多视角查询"的能力。从 MHA→GQA 是平滑折中, MQA (1 套) 就砍得太狠了。

</details>

<details>

<summary>Q2: KV Cache 为什么对推理 (生成) 有效, 对训练没用? </summary>

A: 训练时每个 batch 里所有位置的 Q/K/V 都要一起算、一起反向传播, 缓存 K/V 反而打乱计算图, 毫无意义。推理生成是"逐步、不反向"的, 且前面的 token 固定不变, 它们的 K/V 也就不变——缓存后每步只算新 token 的注意力, 把每步成本从 $O(T^2)$ 降到

O(T)。本质是"自回归生成中，前缀计算天然可复用"。

</details>

<details>

<summary>Q3: MoE 的参数量和计算量为什么不相等？负载均衡损失解决了什么问题？</summary>

A: MoE 把所有专家都"装"进模型，所以参数量很大（一堆 FFN）；但每个 token 只走 top-k 个专家，实际计算量只和 top-k 成正比，两者脱钩——"参数多、算力省"。负载均衡损失解决"路由器把 token 全堆给某几个专家"的问题：它会惩罚"被选过多次的专家"（load 大）和"得分高的专家"（scores.mean 大）的重叠，逼路由器把 token 摊开，避免专家"饿死/撑死"。

</details>

课后作业

完成本章后，去 Assignment 7 完成题 4（repeat_kv）和题 5（SwiGLU）：

[Assignment 7](../../assignments/assignment_7/)

下一步

现代 LLM 的"零件"全部到齐了：BPE tokenizer、RMSNorm、RoPE、GQA + KV Cache、SwiGLU（MoE 可选）。但零件好不等于模型好用——接下来是真正的重头戏：把模型按现代方式训练成助手。预训练 → SFT → DPO，每一步解决什么问题、代码怎么写，下一章见分晓。

[04 — 训练流水线：Pretrain → SFT → DPO](04_training_pipeline.md)

04_training_pipeline

04 — 训练流水线：Pretrain → SFT → DPO

> 零件齐了，该训练了。现代 LLM

不是"一次训练到位"，而是走一条流水线：先预训练成"文档补全器"，再 SFT 成"助手"，最后 DPO 让回答更讨喜。本课把三步从头实现。

前置知识

- 必须掌握：[Part 6 04 章](../../Part6_transformer/tutorial/04_beyond_transformer.md)（预训练 vs 微调、SFT → 奖励模型 → RLHF/PPO 完整概念）

- 建议掌握：[Part 6 01 章](../../Part6_transformer/tutorial/01_data_and_tokenizer.md)（训练循环、AdamW、交叉熵）；[本部分 01 章](01_bpe_tokenizer.md)（BPE tokenizer、chat 格式

<|im_start|></|im_end|>）

- 可选：[本部分 02 章](02_modern_components.md)/[03 章](03_gqa_and_ffn.md)（组件细节——本章按黑盒引用，卡住时回查）

> 这一章是"理论 + 流程"章，代码重点是 SFT 的 loss masking 和 DPO 的参考模型冻结，这是两个最容易出错的环节。

从 Part 6 结束的地方出发

Part 6 最后一章（04）我们画过 ChatGPT 的完整对齐流程：

预训练（文档补全器）→ SFT → 奖励模型 → RLHF/PPO → 问答助手

当时只是"看个图景"。这一章我们把最核心的三步真正写出来：Pretrain → SFT → DPO。DPO 是 2023 年提出的方案，它用简单的分类损失替代了复杂的 RLHF/PPO——这正是我们现在要学的。

完整流水线总览

data/input.txt (莎士比亚, 110 万字符)



① train_tokenizer.py

训练 BPE, 6400 词表 (第 1 章)



② 预训练 (pretrain) |

目标：预测下一个 token |

数据：纯文本（未分角色） |

产出：能"续写"的模型 |



③ SFT（有监督微调） |

目标：只对 assistant 回答算 loss |

数据：问答对（chat 格式） |

产出：会"回答问题"的助手 |



④ DPO（直接偏好优化） |

目标：让回答更"讨喜" |

数据：(好回答, 坏回答) 偏好对 |

产出：对齐人类偏好的模型 |



部署 / 生成 (KV Cache)

每一步的数据不同、损失不同、目标不同，但模型架构从头到尾是同一个（上三章的零件）。这正是现代 LLM 的范式：一个骨架，多阶段训练。

② 预训练（Pretrain）：文档补全器

目标：预测下一个 token

预训练的目标函数和 Part 6 一模一样：给定前 t 个 token，预测第 $t+1$ 个，用交叉熵衡量。唯一区别是 tokenizer 从字符级换成了 BPE（第 1 章）。

```
`
输入 [<|im_start|> ... 一段莎士比亚 ...]
|
预测下一个 token
`
```

- 预训练数据是裸文本：我们直接把 110 万字符的莎士比亚编码成 BPE token 序列去训练，不分 user/assistant。模型在这里学的是“语言的统计规律”——它会续写，但不会回答问题（你问它问题，它可能回你更多问题）。
- ⚠ 预训练产出的模型叫 base model（基座模型），行为不可控。minimind 里这一步叫 `train_pretrain.py`，产出 `pretrain_hidden.pth`。

训练技巧：从 Part 6 的“三行循环”到现代套路

Part 6 的训练循环是“零钱”：

```
`python
for iter in range(max_iters):
    xb, yb = get_batch('train')
    logits, loss = model(xb, yb)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
`
```

现代 LLM 的训练循环加了四个“工程件”，[06_pretrain_pipeline.py](../scripts/06_pretrain_pipeline.py)：

```
`python
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, betas=(0.9, 0.95))

for step in range(max_steps):
    loss = model.compute_loss() # ① 前向 + loss
    loss = loss / accum_steps # ② gradient accumulation
    loss.backward()
    if (step + 1) % accum_steps == 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # ③ clipping
        optimizer.step(); optimizer.zero_grad()
`
```

④ cosine 学习率调度（每个 step 后更新 lr）

```
lr = min_lr + 0.5 (max_lr - min_lr) (1 + math.cos(math.pi * step / max_steps))
for g in optimizer.param_groups: g['lr'] = lr
`
```

1. 混合精度（mixed precision）：`torch.autocast`（即 `torch.cuda.amp`）下，矩阵乘法等计算自动降到 bf16/fp16，梯度用 fp32 累加——显存减半、速度翻倍。bf16 比 fp16 动态范围更大（不容易溢出），是现代 GPU 的默认选择。CPU 上 bf16 指令支持有限，不一定加速。教程脚本以 fp32 为主（保证 CPU 兼容），脚本 [06_pretrain_pipeline.py](../scripts/06_pretrain_pipeline.py) 的 GPU 路径会自动开启

`torch.autocast(device_type='cuda', dtype=torch.bfloat16)`。

2. 梯度累积 (gradient accumulation) : 显存放不下大 batch ? 拆成小 batch 跑 `accum_steps` 次, 梯度累加后再 `step` 一次——等效于大 batch。

3. 梯度裁剪 (gradient clipping) : `clip_grad_norm_(1.0)` 把梯度范数限制在 1 内, 防"梯度爆炸" (语言模型的常见病)。

4. Cosine 学习率调度 + AdamW : 学习率从 `max_lr` 按余弦曲线衰减到 `min_lr` (通常 `min_lr ≈ 0.1 · max_lr`) , 前期大步快走、后期小步精调。

- 对比 Part 6 : 损失函数、模型结构没变, 变的是"怎么更新参数"更稳更快。预训练的核心思想还是那句——预测下一个 token, 压缩语言的结构。
- 预期输出 (CPU 缩小版, BPE、~26M、跑 5000 步左右) : val loss 从初始 ≈ 8.8 (均匀分布的熵 $\ln(6400) \approx 8.76$, 随机初始化略高于它) 一路降到 ≈ 2.0 , 对应 ppl $\approx 7 \sim 12$ ($e^{2.0} \approx 7.4$)。△ BPE 的 loss 数字和 Part 6 字符级的 2.23 不可直接比——词表大了 100 倍、每个 token 携带的信息更多, 初始 loss 自然高得多; 真正可比的是下降趋势和生成质量。训练完生成出来是"伪莎士比亚"。

③ SFT (Supervised Fine-Tuning) : 把补全器变成助手

为什么需要 SFT

预训练模型是"文档补全器"。要它当"助手", 必须用问答格式的数据教它"等一个问题、答一个答案"。这一步就是 SFT (有监督微调) ——minimind 里叫 `train_full_sft.py`。

- 微调之所以样本高效 (几千~几万条数据就能起效), 是因为模型已经通过预训练学会了语言, SFT 只是"重新训练格式与行为": 在预训练的底座上, 用很少的高质量问答数据做微调。

Chat Template : 数据长什么样

SFT 数据长这样 (第 1 章预告的 chat 格式) :

```
<|im_start|>user
What is the capital of France?<|im_end|>
<|im_start|>assistant
The capital of France is Paris.<|im_end|>
```

把这条文本 encode 成 token 序列后, 整个序列一起喂给模型预测下一个 token。对莎士比亚文本做 SFT, 就是把一段段"提问+回答"组装成这种格式 (比如"问: 讲讲 V 夫人的性格。答: <引用原文>...")。

数据怎么构造 ([07_sft_training.py](../scripts/07_sft_training.py)) ——其实就是"字符串拼接 + 编码", 外加记录 assistant 区间 (为后面 masking 做准备) :

```
python
def build_chat_sample(question, answer, tokenizer):
```

1. 拼出 chat 格式的完整文本

```
text = (f"<|im_start|>user\n{question}<|im_end|>\n"
f"<|im_start|>assistant\n{answer}<|im_end|>\n")
```

2. 整体编码成 token 序列

```
input_ids = tokenizer.encode(text).ids
```

3. 记下 assistant 内容在 token 序列里的 [start, end)

```
a_start = len(tokenizer.encode(f"<|im_start|>user\n{question}<|im_end|>\n"
f"<|im_start|>assistant\n").ids)
a_end = len(input_ids) - len(tokenizer.encode("<|im_end|>\n").ids)
return input_ids, a_start, a_end
```

用莎士比亚原文当"答案"：问一句，答一句（我们人为构造的玩具 SFT 集）

```
samples = [build_chat_sample(q, a, tokenizer) for q, a in shakespeare_qa_pairs]
```

- 划重点：**a_start/a_end** 是 token 级的下标，必须用 tokenizer 编码来算（不能直接数字符）。**a_start** 停在 **<|im_start|>assistant\n** 编码完之后的位置——从这里开始才是模型要学的回答。
- 对莎士比亚做 SFT 的一个简单玩法：问题用"关于某角色的提问"，回答直接引用原文段落。数据量不用大，几百条就能让模型"学会问答的格式"。

关键：Loss Masking（只对 assistant 算 loss）

△ 这是 SFT 最容易出错、也最关键的地方。

如果像预训练那样对整条序列算 loss，模型就会学会预测"user 的问题"——但我们根本不在乎模型能不能预测问题（问题是我们给的），我们只在乎它能不能把 assistant 的答案续写对。

做法：把"非 assistant 部分"的标签设成 -100，**F.cross_entropy** 的 **ignore_index=-100** 会自动跳过它们：

```
`python
```

labels 与 input 对齐，先全设成 -100（不计算 loss），再只对 assistant 区间保留

用上面 build_chat_sample 记下的 (a_start, a_end) 填 mask

```
labels = torch.full_like(input_ids, -100)
for s, (a_start, a_end) in enumerate(assistant_spans):
```

关键对齐：位置 t 的标签 = 下一个 token input_ids[t+1]

```
labels[s, a_start - 1 : a_end - 1] = input_ids[s, a_start : a_end] # 监督整段回答（含开头第一个词）
labels[s, a_end - 1] = eos_token_id # 回答结束处学 <|im_end|>
```

```
loss = F.cross_entropy(
    logits.view(-1, vocab_size), # (B*T, vocab)
    labels.view(-1), # (B*T,)
    ignore_index=-100) # ← 跳过所有 -100 的位置
```

- △ 注意这里的偏移细节：**labels[t]** 存的是"位置 t 应该预测出的下一个 token"，所以是 **input_ids[t+1]**。**a_start-1**（**<|im_start|>assistant\n** 的最后一个 token）的标签是回答的第一个词 **input_ids[a_start]**——模型正是从这一刻开始"发言"；接着一路监督到最后一个回答词；**a_end-1** 处再监督

<|im_end|>（让模型学会"说完就闭嘴"）。user 问题、<|im_start|>、格式 token 这些位置全是 -100，被 ignore_index 跳过。

- 顺带一提：预训练也可以不做 masking（整个序列都监督），但 SFT 必须 masking——这正是预训练与 SFT 在损失上的本质区别。

- Loss Masking 的核心：ignore_index=-100 让交叉熵无视 user/格式部分，只训练 assistant 的生成。模型学会的是"看到 user 问题 → 生成 assistant 答案"，而不是"背下所有问题"。

- 这呼应了 Part 6 04 章 SFT 的概念：SFT =

用"问题在上、答案在下"的格式微调。现在你知道了实现它的关键就是 loss masking。

- ⚠ 别犯的错：直接对整个序列算 loss、忘记

masking——那样模型会去"预测问题"，浪费训练信号且行为学歪。

④ DPO（Direct Preference Optimization）：用偏好直接对齐

从 RLHF 出发

Part 6 讲的对齐第三步是 RLHF：先训练一个奖励模型（给回答打分），再用 PPO 强化学习最大化奖励。这条路效果虽好，但工程极重——要训奖励模型、要维护策略/参考/奖励/价值四个网络、要调 PPO 的稳定系数……对小项目不现实。

DPO 的关键洞察：Bradley-Terry 消掉奖励函数

DPO（Direct Preference Optimization, 2023）有一个漂亮的数学洞察：

> 我们真正想要的，是"让好回答概率高、坏回答概率低"。如果能写出这个偏好目标的解析解，就能绕开奖励模型和 PPO，直接用偏好数据算损失。

它用 Bradley-Terry 模型把"哪个回答更受欢迎"建模成排序概率：给定提示 x ，回答 y_w （chosen，更被喜欢）优于 y_l （rejected，更不被喜欢）的概率是

$$P(y_w > y_l | x) = \sigma(r(x, y_w) - r(x, y_l))$$

其中 $r(x, y)$ 是潜在奖励函数， σ 是 sigmoid。DPO

证明：最优策略的解可以把奖励函数"替换掉"——用当前模型和参考模型的对数概率比来表示。最终 DPO loss：

$$L_{DPO}(\pi_\theta) = -E[\log \sigma(\beta \cdot \log(\pi_\theta(y_w|x) / \pi_{ref}(y_w|x)) - \beta \cdot \log(\pi_\theta(y_l|x) / \pi_{ref}(y_l|x)))]$$

↑ chosen 相对参考提升的幅度 ↑ rejected 相对参考提升的幅度

- 拆开看： $\log(\pi_\theta / \pi_{ref})$ 叫隐式奖励——"当前模型比参考模型更看好这个回答多少"。DPO 就是让 chosen 的隐式奖励高、让 rejected 的隐式奖励低，用一个 logsigmoid 把它们塞进同一个分类目标。不需要奖励模型，不需要 PPO。

- ⚠ 其中 β （温度/系数）控制"离参考模型多远"， π_{ref} 是冻结的参考模型（通常是 SFT 完的模型）。参考模型不更新，只是给 chosen/rejected 各自一个"基准概率"，防止模型在优化偏好时把语言能力"忘了"。

代码：DPO 训练

[08_dpo_alignment.py](../scripts/08_dpo_alignment.py) 的核心：

```
`python
```

参考模型 = SFT 权重复制一份，冻结

```
ref_model = build_model(cfg); ref_model.load_state_dict(sft_weights)
for p in ref_model.parameters():
    p.requires_grad = False
```

```
def compute_dpo_loss(policy_logprobs_w, policy_logprobs_l,
ref_logprobs_w, ref_logprobs_l, beta=0.1):
```

隐式奖励：当前模型 - 参考模型

```
reward_w = policy_logprobs_w - ref_logprobs_w # chosen 的  $\log(\pi_\theta / \pi_{\text{ref}})$ 
reward_l = policy_logprobs_l - ref_logprobs_l # rejected 的  $\log(\pi_\theta / \pi_{\text{ref}})$ 
loss = -F.logsigmoid(beta * (reward_w - reward_l)).mean()
return loss
```

每个 (prompt, chosen, rejected) 样本：

policy 模型算 chosen/rejected 的对数概率（可微）

ref 模型用 no_grad 算同样的对数概率（冻结）

```
loss = compute_dpo_loss(...); loss.backward(); optimizer.step()
```

```
,
```

- 数据长这样：`{"prompt": "...", "chosen": "好的回答", "rejected": "差的回答"}`。我们可以在莎士比亚 SFT 模型上人为构造偏好对（比如“回答引用正确的原文” vs “回答胡编”）来演示 DPO。
- 训练时 policy 模型（要训的）要算梯度，ref 模型在 `torch.no_grad()` 下跑、只提供概率基线。这是 DPO 参数更新的标准形态。
- 预期输出：DPO loss 下降、chosen 的回答在隐式奖励上逐渐高于 rejected（可以打印一个 `acc = (reward_w > reward_l).float().mean()` 当“偏好准确率”，从 ~50% 升到 80%+）。

细节：怎么算“一个回答的对数概率”

DPO 的核心原料是 $\log \pi_\theta(y|x)$ ——“模型给回答 y 的平均每个 token 的对数概率”。它不是一次前向就能直接拿到的，要把回答的每个 token 的对数概率取平均（用均值而非累加，避免长回答被不公平地惩罚）：

```
`python
def compute_response_logprobs(model, prompt_ids, response_ids):
    """返回模型对 response 的平均对数概率（DPO 里当‘隐式奖励’用）"""
    seq = torch.cat([prompt_ids, response_ids]) # prompt + 回答 拼起来
    logits = model(seq.unsqueeze(0)).logits # (1, T, vocab)
    log_probs = F.log_softmax(logits, dim=-1)
    shift = len(prompt_ids)
```


位置 t 预测 seq[t+1]：预测 response 每个 token 的分布是 log_probs[shift-1:-1]

```
token_logp = log_probs[0, shift-1:-1].gather(-1, seq[shift:].unsqueeze(-1)).squeeze(-1) # (len(response),)
return token_logp.mean() # 平均 = log π (y|x) / len
```

- ⚠ 两个坑：① 用 **log_softmax** 而不是在 **softmax** 后取 log（数值更稳）；② 只对 response 部分取平均——**shift = len(prompt_ids)** 保证我们只把"回答"的 token 概率加起来，prompt 部分不算（prompt 是给定条件，不参与奖励）。用均值而非累加，是因为不同回答长度不同，累加会让长回答天然有更大的绝对值，均值让长短回答可比。
- SFT 正则防崩坏：纯 DPO
训练有时会让模型语言能力退化（只顾拉开偏好差距，忘了怎么正常说话）。实际脚本 [08_dpo_alignment.py](../scripts/08_dpo_alignment.py) 会在 DPO loss 上加一项 SFT 正则：**loss = dpo_loss + 0.1 * sft_loss**，让模型在优化偏好的同时保持对 chosen 回答的基本语言建模能力。这是 DPO 训练的常见技巧。
- chosen/rejected 各自算一份，再让 policy 和 ref 各算一次，就有了 **compute_dpo_loss** 需要的四个数。policy 那份可微（走反向传播），ref 那份在 **no_grad** 下只当基准。

呼应 Part 6 的 RLHF

- 回顾：RLHF = 奖励模型 + PPO（策略梯度），工程复杂。
- 现在：DPO 用 Bradley-Terry + 参考模型，把"排序偏好"直接变成损失。省掉了奖励模型和 PPO 的稳定性调参。
- 代价：DPO 是"离线"的（只用固定偏好数据集），探索性不如 PPO；但对我们的小模型，DPO 是性价比最高的对齐方式。这也是为什么 minimind 流水线用 DPO 而非 PPO。

完整流水线 + 与 minimind 的对应

阶段	我们（教程脚本）	minimind	数据	产出
Tokenizer	'01_bpe_tokenizer.py'	'train_tokenizer.py'	莎士比亚	6400 词表
预训练	'06_pretrain_pipeline.py'	'train_pretrain.py'	裸文本	'pretrain.pth'（补全器）
SFT	'07_sft_training.py'	'train_full_sft.py'	问答对	'full_sft.pth'（助手）
DPO	'08_dpo_alignment.py'	'train_dpo.py'	偏好对	'dpo.pth'（对齐）

- minimind 官方流水线是 **train_tokenizer → train_pretrain → train_full_sft → train_dpo**，和我们教程的路径一一对应。唯一区别：minimind 用大规模开源数据集（中文+英文），我们用莎士比亚做玩具版——流程同构，规模缩小。

部署/生成：把模型用起来

训练完，部署就是加载权重 + 用 KV Cache 逐步生成（第 3 章的 KV Cache 在这里派上用场）。现代生成还会加几个采样技巧，[05_full_model.py](../scripts/05_full_model.py)：

```
`python
logits = logits[:, -1, :] / temperature # 温度：压低/抬高分布
logits = top_k_filter(logits, top_k=50) # top-k：只留前 50 个候选
```

```
probs = F.softmax(logits, dim=-1)
next_token = torch.multinomial(probs, 1) # 采样（而不是贪心 argmax）
```

- **temperature**：越低越保守（>1 更随机）
- **top_k / top_p**：截掉低概率尾巴，让采样集中在靠谱候选里
- 配合 chat 格式：拼上 `<|im_start|>user\n...<|im_end|>\n<|im_start|>assistant\n` 作 prompt，让模型续写 assistant 的回答，遇到 `<|im_end|>` 停止。

学完本部分你能...

- 画出并讲清 Pretrain → SFT → DPO 完整流水线及每步的目标/数据/损失
- 说出预训练四个技巧（混合精度、梯度累积、梯度裁剪、cosine LR）各自解决什么
- 写出 SFT 的 loss masking (`ignore_index=-100`)，解释"为什么只对 assistant 算 loss"
- 用 Bradley-Terry 讲清 DPO："偏好排序可以直接变成损失，绕开奖励模型和 PPO"
- 写出 DPO loss，说出参考模型冻结的作用（防"忘掉语言"）
- 对照 minimind 的 `train_tokenizer → train_pretrain → train_full_sft → train_dpo`

课后练习

<details>

<summary>Q1: 为什么 SFT 的 loss 要 masking，只对 assistant 部分算？如果对整个序列算会怎样？</summary>

A: 目标不同——我们只关心模型能不能"根据 user 问题生成 assistant 答案"。user

问题是输入、不是输出，让模型预测它是浪费训练信号，还会让模型去"背诵问题"。masking 用 `ignore_index=-100` 跳过非 assistant 位置，让交叉熵只监督答案的生成。如果对整个序列算 loss，模型会同时学习预测问题，行为学歪、收敛也变差。

</details>

<details>

<summary>Q2: DPO 为什么能"不需要奖励模型和 PPO"？"参考模型冻结"是干什么的？</summary>

A: DPO 的数学核心是：在"与参考模型保持 KL 距离"的约束下，最优偏好目标可以解析求解，且解里把奖励函数用「当前模型和参考模型的对数概率比」（隐式奖励）替换掉。所以直接用 (chosen, rejected) 偏好数据 + logsigmoid 就能训练，省掉了奖励模型和 PPO。参考模型（通常取 SFT 权重）冻结、不更新，是给隐式奖励提供"基准"——防止模型为了讨好偏好数据而把语言能力（流畅度、事实）丢掉。

</details>

<details>

<summary>Q3: 如果只做预训练不做

SFT/DPO，模型生成会是什么样？三阶段分别补了什么能力？</summary>

A: 只做预训练：模型是"文档补全器"——给任何前缀都能继续写（风格像训练数据），但你问它问题它可能反问你、或续写新闻稿，行为不可控。SFT 补"对话格式"：学会等一个 user 问题、生成一个 assistant 答案，从"续写"变成"问答"。DPO

补"质量对齐"：让回答更讨喜（更贴偏好），从"会答"变成"答得好"。三者分别对应"会语言 → 会对话 → 会好好对话"。

</details>

课后作业

完成本章后，去 Assignment 7 完成题 6（DPO loss）和题 7（KV Cache）：

完结

恭喜你完成 Part 7（现代 LLM / Minimind）全部四章！

回顾整条路线：Part 1 Bigram → Part 2 MLP → Part 3 BatchNorm → Part 4 反向传播 → Part 5 WaveNet → Part 6 Transformer/GPT → Part 7 现代 LLM。现在你已经从一个"预测名字的 2×2 查表"一路走到从零复现了一个现代 LLM 的全部核心：

6400 BPE 词表 → RMSNorm → RoPE → GQA + KV Cache → SwiGLU → Pretrain → SFT → DPO

> 这和 minimind 的关系：教程用莎士比亚 + CPU 缩小版演示每一步的原理与代码；minimind 用中文/英文大语料 + GPU 把同一条流水线放大到 ~26M 参数真正能聊天的模型。你写的代码和 minimind 的 `model_minimind.py` 结构是同一个。

- 别忘了回到 README 的"演进路线"表格，把每个零件再对照一遍。
- 动手做 [Assignment 7](../assignments/assignment_7/)，然后可以去读 minimind 源码，你会发现全都能看懂了。
- 毕业下一步：跑 [scripts/09_eval_demo.py](../scripts/09_eval_demo.py) 做三阶段验收（Base/SFT/DPO 生成对比 + ppl），然后按 [05 — 复现 minimind 毕业指南](05_reproduce_minimind.md) 在真实中文数据上跑官方仓库。

[← 上一章：Part 6 Transformer](../Part6_transformer/tutorial/README.md)

05_reproduce_minimind

05 — 复现 minimind 毕业指南：从本课脚本到官方仓库

- > 前面 4 章我们用自包含缩小版复现了 minimind 的组件与三阶段训练。这一章是"最后一公里"：
- > 只靠本指南（不读原仓库 README），在真实数据上跑通官方的
- > `train_tokenizer` → `train_pretrain` → `train_full_sft` → `train_dpo`，并用 `eval_llm.py` 交出一个具体数字。
- >
- > 内容基于 minimind master（2026-08 核对，Apache-2.0）。上游更新较快，若对不上以原仓库为准。
- > 验证边界（诚实声明）：本指南的事实（脚本路径、参数默认值、数据文件名与格式、
- > 配置字段、时长报价）已逐一对照 minimind master 源码核验；但端到端跑通需要下载
- > 2.9GB 数据 + 单卡数小时，未纳入课程自动验证。请按第 5 节成本现实自行执行，
- > 卡住时优先对照原仓库 `trainer/` 里的同名参数。

学习目标

完成本章后，你将能够：

- 跑通官方 minimind 四件套（tokenizer → pretrain → SFT → DPO）并交出 `eval_llm.py` 数字
- 对照课程缩小版脚本与官方 `trainer` 的字段级映射（知道每个参数放大成什么）
- 解释长上下文外推四方案（naive/PI/NTK/YaRN）的 ppl 与 needle 检索实测排序

- 评估 长上下文能力的三层验证法（NIAH 冒烟 / RULER 类合成套件 / 真实任务榜单）

前置知识

- 必须掌握：Part 7 的 01-04 章（尤其 04 章三阶段训练）——本指南假设你理解每个阶段在做什么、为什么这么配超参
- 建议掌握：[Part 8 02 章](../Part8_post_training/tutorial/02_sft_and_chat.md) 的 SFT 数据格式（官方 SFT 数据同构）；[docs/datasets.md](../docs/datasets.md) 的语料下载方式
- 可选：一块 $\geq 8\text{GB}$ 显存的 NVIDIA GPU（租的也行，见"成本现实"；进阶实验的脚本 11 CPU 也能跑、脚本 13 GPU 更快）

第 0 步：结构对照——课程脚本 $\leftrightarrow$ 官方文件

官方仓库的训练脚本在 [trainer/](#) 目录（不在根目录，别找错）：

	关键差异			
tokenizer.py	官方用 HF `tokenizers` 的 ByteLevel BPE，词表 6400，在 sft 语料上训练			
minimind.py	架构同构（RMSNorm+RoPE+GQA+SwiGLU+tie）；官方多 q/k RMSNorm、flash-attn、YaRN 选项			
retrain.py	官方吃 `pretrain_t2t_mini.jsonl`（中文问答对）			
ll_sft.py	官方 loss mask 只监督 `<`	im_start	>assistant` 到 `<`	im_end
dpo.py	官方 beta=0.15、lr=4e-8（"建议 $\leq 5e-8$ 避免遗忘"）			
	官方是交互式 chat；榜单跑 lm-evaluation-harness			

第 1 步：环境与数据下载

```
`bash
git clone https://github.com/jingyaogong/minimind && cd minimind
pip install torch transformers datasets tokenizers modelscope accelerate`
```

数据放 [./dataset/](#)（ModelScope 国内快，HuggingFace 可设镜像）：

```
`bash
官方推荐最小组合（共 ~2.9GB）
```

```
modelscope download --dataset gongjy/minimind_dataset \
pretrain_t2t_mini.jsonl sft_t2t_mini.jsonl dpo.jsonl --local_dir ./dataset
```

HuggingFace（国内可加 export
HF_ENDPOINT=https://hf-mirror.com）

下载中断重跑同一条命令即可续传（modelscope/hf
均支持断点续传）

文件	大小	用途	每行格式 (jsonl)
'pretrain_t2t_mini.jsonl'	1.2GB	预训练	`{"text": "如何才能摆脱拖延症？..."}`
'sft_t2t_mini.jsonl'	1.6GB	SFT（多轮对话）	`{"conversations": [{ "role": "user", "content": "你好"}, { "role": "assistant", "content": "你好！"}]}`
'dpo.jsonl'	53MB	DPO	`{"chosen": [{ "content": "Q", "role": "user"}, { "content": "好回答", "role": "assistant"}], "rejected": [{...坏回答...}]}`

> 注意：网上老教程里的 `pretrain_hq.jsonl` / `sft_512.jsonl` 是已废弃的旧文件名，现在都是 `_t2t` 命名。

第 2 步：配置对照——把"课程缩小版"放大回官方规模

配置	hidden	layers	q/kv heads	vocab	rope θ	参数量
本课 CPU 版	64	2	4/2	字符级 256	1e4	~0.3M
本课 GPU 模板	768	8	8/4	6400	1e4	~64M
minimind2-small（推荐起点）	**512**	**8**	**8/2**	6400	**1e6**	**26M**
minimind-3	768	8	8/4	6400	1e6	64M
minimind-3-moe	768	8	8/4（4 专家 top-1）	6400	1e6	198M-A6xM

- > $\triangle$ 两个容易看漏的字段：`intermediate_size` 官方公式 $\text{int}((\pi \cdot \text{hidden}/64)+0.5) \cdot 64$ （512→2432）；
- > 官方 `max_position_embeddings=32768`、`rms_norm_eps=1e-6`、`tie_word_embeddings=True`。
- > 26M 的 kv_heads 是 2（不是 4）——GQA 压得更狠。

配置变化的因果（面试常问"参数放大 10 倍，超参怎么跟着动"）：

- lr 降：模型越大梯度噪声越小但发散风险越大，26M 用 5e-4，百 M 降到 ~3e-4；
 - batch（×梯度累积）升：更大 effective batch 稳住大模型训练（官方 pretrain effective = 32×8=256）；
 - seq_len 升：预训练 340 → SFT 768 → DPO 1024，随阶段需要的上下文变长；
 - warmup/cosine：官方把 schedule 封装成 cosine 从 $1.0 \times \text{lr}$ 衰减到 $0.1 \times \text{lr}$ ，无独立 warmup
- 参数——小模型短训练可以直接不 warmup。

第 3 步：四阶段训练（官方默认超参，可直接抄）

`'bash`

① 分词器（官方不建议重训，直接用仓库自带 `minimind_tokenizer`；想重训：）

`python trainer/train_tokenizer.py`

② 预训练：`epochs=2, bs=32, accum=8(effective 256), lr=5e-4, seq=340, bf16`

`python trainer/train_pretrain.py --epochs 2 --batch_size 32 --accumulation_steps 8 \`
`--learning_rate 5e-4 --max_seq_len 340 --data_path ./dataset/pretrain_t2t_mini.jsonl \`
`--save_weight pretrain --dtype bfloat16`

③ SFT：epochs=2, bs=16, lr=1e-5, seq=768（从 pretrain 权重续）

```
python trainer/train_full_sft.py --epochs 2 --batch_size 16 --learning_rate 1e-5 \
--max_seq_len 768 --data_path ./dataset/sft_t2t_mini.jsonl \
--from_weight pretrain --save_weight full_sft
```

④ DPO：epochs=1, bs=4, lr=4e-8(≤5e-8!), beta=0.15, seq=1024（从 SFT 权重续）

```
python trainer/train_dpo.py --epochs 1 --batch_size 4 --learning_rate 4e-8 \
--max_seq_len 1024 --data_path ./dataset/dpo.jsonl --beta 0.15 \
--from_weight full_sft --save_weight dpo
```

产物在 ./out/（如 pretrain_512.pth → full_sft_512.pth → dpo_512.pth）。

第 4 步：验收——交出具体数字

```
`bash
```

交互式对话冒烟测试

```
python eval_llm.py --weight_mode 1 --load_weight 1 --hidden_size 512 # 加载 out/dpo_512.pth
```

榜单（官方用 lm-evaluation-harness）

```
lm_eval --model hf --model_args pretrained=<你的transformers格式权重> \
--tasks ceval cmmlu arc_easy piqa --batch_size 8
```

预期行为对照表（判断自己训没训对）：

阶段	问它一句话，应该...
pretrain 后	输出**流利中文但答非所问**（续写"问句"而不是回答）
SFT 后	**能按一问一答格式说话**，内容可能仍简单/有错
DPO 后	风格更"讨喜"，长答案结构更好（幅度不大，lr 极小是故意的）

参考量级（官方 minimind-3, lm-eval-harness）：ceval 24.89 / cmmlu 25.38 / arc_easy 28.49 / piqa 50.65——小模型在多选任务上接近随机（25-30%）是正常的，别慌，看相对变化而不是绝对分数。

第 5 步：成本现实（没有卡也能跑）

项	数字
官方实测（RTX 3090 单卡，bf16，dense 64M）	pretrain ≈ 1.21h + SFT ≈ 1.10h ≈ **2.3h**，市价约 ¥3
显存	26M/64M bf16 + effective batch 256 → **<24GB**，3090/4090 单卡即可
租卡	AutoDL / 智星云 / 仙宫云等按时租用 3090/4090，¥1-2/小时档；跑完全流程一杯奶茶钱

项	数字
数据	2.9GB, ModelScope 国内直连一般 10-30 分钟

> 建议：先租 2 小时把 ②③ 跑通看到模型说话，再决定要不要补 ④——正反馈最快。

第 6 步：进阶实验（面试加分项）

实验 1：RoPE 长上下文四件套（ppl 版）

RoPE 长上下文四件套（minimind 内置 `inference_rope_scaling` 选项）：

	做法	关键数字
外推)	角度表算到新长度，什么都不改	训练外的旋转角全是分布外 → 外推区 ppl 明显劣化
Interpolation	位置 $m \rightarrow m/s$ 压进训练范围	Llama-1 7B 2k→32k 只需 ~1000 步微调（论文实验口径）；高频维度被过度压缩 → 零样本
	改 base： $\theta' = \theta \cdot s^{(dim/(dim-2))}$	高频几乎不动（局部序保留），小倍数可近零样本外推 ~2×
	逐维 ramp 混合 PI/NTK + 注意力温度 $\sqrt{(1/t)} = 0.1 \cdot \ln(s) + 1$	7B 128k 模型 400+200 步微调（s=16 用 400 步到 64k、s=32 再加 200 步），比 PI 省 ~1/2

- > YaRN 三部件（实现对照 HF `modeling_rope_utils.py`，论文 [2309.00071](https://arxiv.org/abs/2309.00071)）：
- > ① `find_correction_dim` 反解"在训练长度内转 32 圈 / 1 圈"的维度边界；
 - > ② 逐维 ramp：高频维（短波长）原样外推、低频维（长波长）全插值、中间线性过渡；
 - > ③ 温度：softmax 前给 q 乘 $\sqrt{(1/t)} = 0.1 \cdot \ln(s) + 1$ 微微锐化注意力（论文/HF 官方做法是 $\sqrt{(1/t)}$ 同时乘 q 、 k ，等价于 $\logit \times 1/t$ ；本课脚本只乘 q ，玩具尺度上两者几乎不可区分）。
 - > 命名对照：论文 Eq.11 只用无下标的 $\alpha=1 / \beta=32$ ；HF 把 β 重命名为 `beta_fast=32`（高频维边界）、 α 重命名为 `beta_slow=1`（低频维边界）——取值与边界一一对应，只是多了 fast/slow 后缀。

亲手实验：跑本课 `[scripts/11_rope_scaling.py](../scripts/11_rope_scaling.py)` ——
同一模型只换位置方案，实测"训练 128 → 推理 256"（s=2）的外推 ppl（RTX 4090 / torch 2.6.0 / 2026-09 实测，CPU 复跑趋势一致；PI 为连续插值实现——位置 m/s 是小数，角度表相邻行线性插值，若直接整数截断会让相邻 token 位置重合、ppl 假性变差）：

```
`text
方案 ppl@ctx=128（训练内） ppl@ctx=256（外推）
① naive（直接外推） 5.00 6.37
② PI（位置 ÷ 2，连续插值） 14.06 13.35
③ NTK（base × s^(d/(d-2))) 5.08 5.20
④ YaRN（ramp+温度1.069） 5.27 5.05`
```

三个读数：训练内 PI 崩到 14+（它把见过的位置也压掉一半，零样本等于换位置分布——"PI 必须配微调"不是论文套话，是实测）；外推区 naive +1.4 劣化；YaRN 外推区最优（5.05，甚至低于自己训练内的 5.27——插值把低频维压回训练范围，抵消了外推噪声）。

实验 2：迷你 RULER（needle 检索版）——ppl 不等于"记得住"

- > 衔接：实验 1 的 ppl 只测"读得顺不顺"（下一 token 概率），不测"从 2 万字里取回某条具体信息"。
- > 长上下文的实用能力是后者——这正是 RULER 论文的核心主张。

跑本课 `[scripts/13_long_context_eval.py](../scripts/13_long_context_eval.py)`：

合成 KV 检索任务 ("a3,b7,...,a3,b7,...?b → 7", 字典 21 对、每对出现两次、query 问一个 key 的值), 同一模型 (训练长度 128) 在 {128, 256, 512} 三档 × 四方案上的 needle 准确率 (RTX 4090 / torch 2.6.0 / 2026-09 实测, 随机猜 = 0.100) :

```
`text
ctx (s) naive pi ntk yarn
128 (s=1) 1.000 1.000 1.000 1.000 ← 训练内四方案数学上等价 (sanity check)
256 (s=2) 0.922 1.000 1.000 1.000
512 (s=4) 0.422 0.500 0.891 1.000 ← naive 崩、PI 零样本不稳、yarn 满分`
```

与实验 1 互补且互相印证: ppl 里 $yarn \leq ntk < naive$, 检索准确率里 $yarn \geq ntk \gg naive$ 。
曲线图存 [scripts/output_long_context.png](#)。

> △ 该脚本训练段在 CPU 上约 45 秒 (实测 GPU 约 5-15 秒, 随卡与 autotune 波动): 检索电路 (归纳头) 不是渐进变好,
> 而是训练到 ~2000 步"顿悟"式出现 (loss 长平台后 accuracy 0.3→1.0 跳变), 步数不能再砍。

为什么 NIAH 不够: 长上下文要怎么评测?

大海捞针 (NIAH, 在长文里藏一句话再问出来) 曾是最流行的长上下文演示, 但它会严重高估能力:

- [RULER (arXiv 2404.06654)] (<https://arxiv.org/abs/2404.06654>): 评测了 17 个声称 $\geq 32K$ 上下文的模型——它们在朴素 NIAH 上都接近满分, 但在更难的变体 (多针、多跳追踪、聚合) 上大幅掉点, 只有一半能在 32K 长度上维持满意表现。NIAH 只代表"最表层的一种长上下文理解"。
- [LongBench v2 (arXiv 2412.15204)] (<https://arxiv.org/abs/2412.15204>): 503 道 8k~2M 词的多选题, 直接作答的最好模型只有 50.1% 准确率 (o1-preview 靠更长推理链到 57.7%, 15 分钟限时的人类专家 53.7%) ——真实长文理解远未解决。
- [HELMET (arXiv 2410.02694)] (<https://arxiv.org/abs/2410.02694>): 主张用"现实任务" (代码库、多轮对话、长依赖阅读理解等) 取代纯合成 NIAH 来衡量有效上下文。

面试答法: "长上下文能力要分三层验证——① NIAH 只能当冒烟测试; ② 合成任务套件 (RULER: needle/多跳/聚合, 我们脚本 13 是其迷你版) 量'有效上下文长度'; ③ 真实任务榜单 (LongBench v2 / HELMET) 看落地。只报上下文窗口大小的营销数字, 一测一个不吱声。"

MoE 负载均衡: 跑本课 `[scripts/10_moe_load_balance.py](../scripts/10_moe_load_balance.py)`, 直观看到"没有 aux loss → 专家贫富分化; 加了 $\alpha \cdot N \cdot \sum f_i \cdot P_i \rightarrow$ 负载拉平"。官方 minimind 用 `router_aux_loss_coef=5e-4` (Switch 论文推荐 $\alpha=0.01$, DeepSeek-V3 已改用无 aux loss 的 bias 法)。

进阶实验: 概念检验与动手实践

<details>

<summary>进阶 Q1: 为什么 YaRN 在外推区 (5.05) 的 ppl 反而低于自己训练内 (5.27)? </summary>

A: 训练内的 5.27 含"温度 1.069 锐化 + 大部分维对被 (部分或完全) 插值 (自检 2: 15/16 个维对落在插值区间, 其中约 10 个全插值、5 个部分插值)"的轻微扰动; 外推时, naive 的痛点是低频维的旋转角超出训练范围 (分布外), YaRN 把这些维插值压回训练范围, 消掉的噪声比引入的扰动多, 于是出现"外推更好"的反直觉读数。这也提醒: ppl 对比要在同一方案自身的训练内/外推两栏看, 跨方案的绝对值受各自扰动影响。

</details>

<details>

<summary>进阶 Q2: PI 为什么"训练内也崩"? NTK 为什么不崩? </summary>

A: PI 把所有位置 $m \rightarrow m/s$: 模型在训练长度内见过的位置分布被整体压掉一半 (训练时位置 64 对应的角度, 推理时出现在位置 32), 等于换了一个位置分布——零样本必然掉点, 所以论文都配微调 (~1000 步)。NTK 只改 base、按频率分摊压缩: 高频维 (决定相邻 token 局部顺序) 几乎不动, 低频维分担压缩, 位置-角度映射保持连续单调, 训练内的行为几乎不变。

</details>

<details>

<summary>进阶 Q3: 脚本 13 的检索任务为什么把字典设计成"每对出现两次"? </summary>

A: 这是把任务改造成"归纳头 (induction head) 电路"可解的形式——第一次出现 $[k] \rightarrow v$ 建立关联, 第二次出现 $[k]$ 时模型靠"找上一次 k 后面跟了什么"来答题 (+1 偏移的归纳电路), 比"死记 21 对字典"更容易在几千步内涌现, 也更贴近真实长上下文的"检索"用法 (RULER 的多键变体同理)。若每对只出现一次, 模型只能靠记忆, 小模型上收敛极慢甚至不收敛。

</details>

动手实践：把外推倍数拉到 $s=4$

任务：修改 `[scripts/1l_rope_scaling.py](../scripts/1l_rope_scaling.py)` 的外推档位 (`ctx_eval` 从 256 改为 512, $s=4$)，重跑并记录四方案的 ppl 排序。

验收标准：

- [] `ppl@512` 排序仍满足 $yarn \leq ntk < naive$ (PI 允许仍最差)
- [] YaRN 温度随 $s=4$ 变为 $0.1 \cdot \ln(4)+1 \approx 1.139$ (输出里有打印)
- [] 能用一句话解释"为什么 s 越大, naive 与其他三者的差距越大" (低频维分布外的比例上升)
- [] (进阶) 再跑 `[scripts/13_long_context_eval.py](../scripts/13_long_context_eval.py)`

对照 `ctx=512` 档的 needle 准确率, 验证"ppl 排序与检索能力排序一致"

面试直通车

<details>

<summary>Q1: 你复现过 minimind? 三个阶段的 lr 差一个数量级, 为什么? </summary>

A: pretrain $5e-4$ (随机初始化, 需要大步子从零学统计规律); SFT $1e-5$ (预训练权重已很好, 大 lr 会灾难性遗忘语言能力, 只微调"格式与指令遵循"); DPO $4e-8$ (对齐阶段动的是偏好分布, lr 稍大就会把 SFT 能力打崩——原仓库注释明说" $\leq 5e-8$ 避免遗忘")。核心: 越靠后的阶段, 改动越"表面"、越需要保住底层能力, lr 递减 1-2 个数量级是通用规律。

</details>

<details>

<summary>Q2: SFT 的 loss mask 具体遮哪里? 为什么不遮 prompt? </summary>

A: 只对 `<|im_start|>assistant\n` 到 `<|im_end|>` 之间的 token 算 CE, prompt 与 padding 全部置 -100。因为训练目标是"学会回答"而不是"学会复述问题"——不 mask 的话模型会浪费容量去拟合用户输入的分布, 还会学会自问自答的怪格式。

</details>

<details>

<summary>Q3: DPO 的 $\beta=0.15$ 意味着什么? 调大调小会怎样? </summary>

A: β 是"离参考模型的信任度": $\beta \cdot (\Delta \pi - \Delta \text{ref})$ 过 sigmoid。 β 大 $\rightarrow$ 更信任参考模型、更新保守, 不容易遗忘但偏好学得慢; β 小 $\rightarrow$ 更激进, 偏好摆动大、容易退化 (verbose/repetition)。官方 0.15 配 lr $4e-8$ 是小模型上稳定的一档。

</details>

验收标准

- [] 只靠本指南，在 minimind2-small (26M) 上跑完 ②③④，out/ 有三个权重
- [] eval_llm.py 加载 DPO 权重能对话，且行为符合"预期行为对照表"
- [] 能不看资料说出：t2t 数据格式、loss mask 位置、四阶段超参及"为什么这么定"
- [] (加分) 跑了 10_moe_load_balance.py，能解释 α 的作用曲线
- [] (加分) 跑了 11_rope_scaling.py + 13_long_context_eval.py，能解释四件套的 ppl/准确率排序、YaRN 温度因子 $\sqrt{(1/t)=0.1 \cdot \ln(s)+1}$ ，以及"为什么 NIAH 不够"

相关资源

- [jingyaogong/minimind](https://github.com/jingyaogong/minimind) (Apache-2.0)
- [数据集 ModelScope](https://www.modelscope.cn/datasets/gongjy/minimind_dataset/files) / [HF 合集](https://huggingface.co/collections/jingyaogong/minimind)
- [YaRN (arXiv 2309.00071)](https://arxiv.org/abs/2309.00071) · [Position Interpolation (2306.15595)](https://arxiv.org/abs/2306.15595)
- [RULER 长上下文评测 (2404.06654)](https://arxiv.org/abs/2404.06654) · [LongBench v2 (2412.15204)](https://arxiv.org/abs/2412.15204) · [HELMET (2410.02694)](https://arxiv.org/abs/2410.02694)
- [Switch Transformer (2101.03961)](https://arxiv.org/abs/2101.03961) · [DeepSeek-V3 aux-free balancing (2408.15664)](https://arxiv.org/abs/2408.15664)

[← 上一章：训练流水线](04_training_pipeline.md) | [Part 7 README](README.md) |
[选修下一章：注意力演进 MLA/NSA →](06_attention_mla_nsa.md)

06_attention_mla_nsa

06 — 注意力演进：MLA 与原生稀疏注意力（NSA）

- > RoPE/GQA 之后，注意力机制还在演进。两条主线都来自 DeepSeek：
- > MLA（纵向压缩 KV 缓存）与 NSA（横向稀疏化注意力计算）。
- > 本章配 [scripts/12_mla_nsa_accounting.py](./scripts/12_mla_nsa_accounting.py)
- > （CPU 5 秒，账本+三分支机制全手写）。论文阅读实战：
- > [DeepSeek-V2 论文](https://arxiv.org/abs/2405.04434) § MLA 节与
- > [NSA 论文](https://arxiv.org/abs/2502.11089) § 2-3——用 docs/paper_reading_guide.md 的
- > 五步法读（符号表→直觉→边界→单调性→数值验证）。

前置知识

- 必须掌握：[03 章](03_gqa_and_ffn.md)（GQA 与 KV Cache——MLA 的对照面、NSA 的稀疏对象）
- 建议掌握：[02 章](02_modern_components.md)（RoPE——MLA 的"解耦 RoPE"是它的直接后续）
- 可选：DeepSeek-V2/V3 技术报告的注意力章节（先学课程版再看原文更顺）

1. MLA：把 KV 压成一个 latent 向量

GQA 靠"少几个 KV 头"省缓存；MLA 换一条路——低秩压缩：

MHA/GQA：每个 token 存 $n_{kv_heads} \times head_dim$ 的 K 和 V
MLA：每个 token 存 1 个 c_KV 向量 (kv_lora_rank 维)，
注意力时用上投影矩阵还原各头的 K/V

DeepSeek-V2 的真实数字（32 层/128 $head_dim$ /seq 2048/fp16，脚本 Part A 实测）：

MHA : 1.07 GB
GQA(kv8) : 0.27 GB
MLA : 0.08 GB (KV 缓存降至 MHA 的 7.0%；逐 token 复算与公式一致)

- 解耦 RoPE 的坑：RoPE 是位置相关的旋转矩阵，会破坏低秩压缩的"矩阵吸收"（ W_{UK} 无法吸收进 W_{UQ} ）。MLA 的方案：给每个 token 额外携带一个小的**位置专用 key** ($k \wedge R$)，与内容 latent 分离，注意力得分两部分相加。
- 矩阵吸收：推理时将 W_{UK} 吸收进 W_{UQ} ，decode 直接从压缩缓存计算注意力——无需还原完整 K。这是 MLA 工程实现的关键一步。

2. NSA：三支原生稀疏注意力

NSA（2502.11089，ACL'25 最佳论文）的核心主张：从预训练起就原生训练稀疏模式（而非推理期后置近似），三支并行：

分支	做什么	成本
压缩分支	每块平均成摘要 token，提供全局粗读	n_blocks
选择分支	由压缩分数选 top_k 关键块，块内精细注意力	top_k 块
滑动窗口	只看最近 window 个 token	window

三支输出经可学习门控线性组合。脚本 Part B 用固定等权门控的最小实现展示三支机制（等权门控下与全注意力的 $\max diff \approx 2.3$ ，门控可学习后收敛）。

3. MLA vs GQA vs NSA：三条压缩路线的对照

维度	GQA	MLA	NSA
压缩对象	KV 头数	KV 缓存维度（低秩）	注意力计算的模式（稀疏化）
缓存降幅	线性（按头数）	**按 latent 维大幅降**	不减缓存，减 FLOPs
训练方式	端到端	端到端	**原生**稀疏训练
硬件亲和	—	矩阵吸收优化	GPU kernel 对齐

学完本部分你能...

- 算出 MHA/GQA/MLA 的 KV 缓存字节数（逐 token 复算与公式一致）
- 解释 MLA 的解耦 RoPE 与矩阵吸收

- 画出 NSA 三支的分工与门控融合
- 说出"原生可训练稀疏"与"推理期后置近似"的区别

课后练习

<details>

<summary>Q1: MLA 为什么不能把 RoPE 也压进 latent ? </summary>

A: RoPE 是位置相关的旋转矩阵——对每个 token 施加不同的旋转。低秩压缩要求 K/V 能被同一个上投影矩阵还原，但旋转后的 K/V 位置相关、投影矩阵无法统一吸收（W_UK 被 R_t 打断）。MLA 的解法：位置部分单独走一个小的 RoPE 通路（ $k \times R$ ），与内容 latent 解耦。

</details>

<details>

<summary>Q2: NSA 的"原生可训练"为什么比"推理期后置稀疏化"效果好 ? </summary>

A: 后置稀疏化（如 H2O/StreamingLLM）是在训练好的稠密模型上近似——模型从未见过稀疏模式，误差随稀疏度放大。原生训练让模型学会"在稀疏模式下表现最好"，且配合硬件对齐 kernel（NSA）实际速度反超稠密。类比：LoRA vs 后置剪枝的关系。

</details>

动手实践：改 latent 维度，画 KV 显存-精度权衡曲线

任务：修改 [scripts/12_mla_nsa_accounting.py](../scripts/12_mla_nsa_accounting.py) 中 MLA 的 `kv_lora_rank` (latent 维度，脚本默认 512)，在 {32, 64, 96, 128} 四档下重跑，记录每档的 KV cache 字节数，画"KV 字节数 - latent 维度"单变量折线图。

验收标准：

- [] KV 字节数随 latent 维度线性增长（每档打印值可对上 $kv_lora_rank \times dtype$ 字节数的账本）
- [] $kv_lora_rank=128$ 与 GQA 的 KV 字节数对比方向正确（谁省谁费说得清）
- [] 能用一句话解释"为什么维度越小越省、但小到某个点精度会崩"（信息瓶颈）

课后作业

完成 [scripts/12_mla_nsa_accounting.py](../scripts/12_mla_nsa_accounting.py) 的两个 Part 后，回答：

1. 把 `kv_lora_rank` 从 512 改成 256，MLA 的缓存变多少？质量预期如何变化？
2. 把 NSA 的 window 从 4 改成 8，三支的相对贡献怎么变？

下一步

Part 11 的 GRPO 是单轮 RLVR——Agentic RL 把它扩展到多轮工具调用与长程任务（→ Part 17 Agentic RL，与本章同属 DeepSeek 架构创新的延伸）。

[Part 17 Agentic RL](../Part17_agentic_rl/tutorial/README.md)

[← 上一章：复现 minimind 毕业指南](05_reproduce_minimind.md) | [下一站：Part 8 后训练全流程 →](../Part8_post_training/tutorial/README.md)（本章为 Part 7 选修章：想继续架构线可读 [Part 17 Agentic RL](../Part17_agentic_rl/tutorial/README.md) 的延伸）